EasyCrypt tricks

Every listing below is a complete file from this folder. Each compiles on its own against the standard library, so any of them can be copied out, run, and changed.

```
for f in *.ec; do easycrypt compile "$f" || echo "FAILED: $f"; done
```

Every file was checked with EasyCrypt r2026.07-78 and with the main branch of October 2026 (r2026.10-1), each time with all provers and with Z3 alone (`easycrypt compile -p Z3`). The pitfalls in Section 16 were observed with r2026.07-78.

Contents

1 Rules of thumb **2**

2 Reading the rules **2**

3 `conseq` **2**

4 `call` **4**

5 `exlim` and `seq` **6**

6 Probabilities **8**

7 Up to bad **10**

8 Dropping one-sided calls **13**

9 `rnd`: coupling samplings **15**

10 A sample the run does not read **17**

11 A probability as a spec, and averaging **19**

12 Laws, and equivalences `rnd` cannot prove **21**

13 Loops **23**

14 `fel`: a charge per oracle call **26**

15 `ehoare`: expectations **27**

16 Pitfalls **30**

17 Seeing the goals from the command line **31**

1 Rules of thumb

- `exlim` **binds the value at the start of the remaining program.** That is the value in the current precondition. If the program changes it before the point where you need it, cut the program first with `seq` or `sp`. `exlim e ⇒ v` is the same as `exists* e; elim* ⇒ v`.
- **Calls are processed from the end.** When dropping one-sided calls, handle the last one first.
- **An equiv cannot relate a side to its own past.** Fix the old value with `exlim` and add a hoare triple for that side with `conseq E H1 H2`.
- `byequiv lemma` **needs an exact match.** Write the spec you want in `byequiv (: P ⟹ Q)` and then `conseq lemma`. If the games are on the other sides, use `symmetry` first.
- **An up-to-bad invariant must include the equality of the flag**, or nothing says the left flag is still down while the right one is.
- **A fact stated as** $\Pr[\ldots] = 1$ **becomes a spec with** `bypr`**.** For a phoare spec, `bypr ⇒ &m <-; exact (h &m)` is enough. For a hoare spec, go through `Pr[mu_not]` as in Section 6.
- **A probability is a spec.** `bypr`, `byequiv` and `proc true` turn `Pr[G.main(x0) @ &m : res]` into a phoare that a `call` can use. That is how a bound for every value becomes a bound for a random one, and how a theorem about a random key, world or oracle gives one fixed value (Section 11).
- **A sample the run never reads is drawn after the run.** Swap it behind the call; the chance that it lands in what the run outputs is then the mass of that set (Section 10). First show that the run really does not read it.
- **Symmetry is a coupling of the game with itself.** Swap the samplings on one side so that the two hidden values cross (Section 9).
- **Same law but no bijection:** `bypr (res1) (res2)`, then one `byphoare` per side (Section 12).
- **Reason about a loop once.** Prove its functional spec, a hoare triple and losslessness, and use it on each side of an equiv alone. The two programs need not run in step (Section 13).
- **A constant charge per call is** `fel`**; a charge that depends on the state is an ehoare potential**, an invariant of every oracle call (Sections 14 and 15).
- **ehoare cannot use a hoare spec.** Prove the ehoare for a reference program that computes the function, the equiv to it, and apply `conseq E H` (Section 15).

2 Reading the rules

Each section below states, as inference rules, the program-logic rule behind the tactics it shows. The rules are simplified to the shape the tactic uses, and they are read bottom-up, because the tactic turns the goal below the line into the goals above it.

- $\models c_1 \sim c_2 : \Phi \Rightarrow \Psi$ is a pRHL judgement. From any two memories related by Φ, the output distributions of c_1 and c_2 are related by Ψ, which means there is a coupling of them whose support lies in Ψ.
- Φ and Ψ relate two memories. For a formula P over one memory, $P\{1\}$ reads it in the left memory and $P\{2\}$ in the right one, and $=\{x\}$ abbreviates $x\{1\} = x\{2\}$.
- $\models c : P \Rightarrow Q$ is a Hoare judgement, saying that every terminating run of c from a memory in P ends in Q. It says nothing about termination.
- $\models c : P \Rightarrow Q\ [=p]$ is a probabilistic Hoare judgement, saying that from every memory in P the probability that c ends in Q is exactly p. With $p = 1$ it gives both termination and the postcondition.
- $\mathrm{mod}(c)$ is the set of variables c may write. When a rule quantifies over values for them, EasyCrypt shows the quantified names as `result_L`, `0_R`, `S_R` and so on.

3 `conseq`

- `conseq H` applies a spec lemma and leaves two side goals, the precondition implication and the postcondition implication.
- `conseq (: _ ⟹ Q)` changes the postcondition to `Q` and keeps the precondition. Use it to strengthen the postcondition to the equalities that `sim` proves.
- `conseq E H1 H2` conjoins a hoare triple for each side onto a two-sided equiv. An equiv relates the two sides to each other, never a side to its own initial value, so facts like "the left counter went up by one" have to come from a one-sided hoare triple.
- `(_: true ⟹ true)` in place of `H1` or `H2` means that nothing is added on that side. It becomes a trivial goal.

- `conseq ll H` turns losslessness and a hoare triple into a phoare spec with probability 1, which is what `call{1}` needs.
- `conseq H1 H2` on a hoare goal conjoins the postconditions of two hoare triples.

$$\frac{\Phi \Rightarrow \Phi' \qquad \models c_1 \sim c_2 : \Phi' \Rightarrow \Psi' \qquad \Psi' \Rightarrow \Psi}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi}\ \texttt{conseq}$$

$$\frac{\models c_1 \sim c_2 : \Phi' \Rightarrow \Psi' \qquad \models c_1 : P_1 \Rightarrow Q_1 \qquad \models c_2 : P_2 \Rightarrow Q_2 \\ \Phi \Rightarrow \Phi' \wedge P_1\{1\} \wedge P_2\{2\} \qquad \Psi' \wedge Q_1\{1\} \wedge Q_2\{2\} \Rightarrow \Psi}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi}\ \texttt{conseq E H1 H2}$$

$$\frac{\models c : \mathit{true} \Rightarrow \mathit{true}\ [=1] \qquad \models c : P \Rightarrow Q}{\models c : P \Rightarrow Q\ [=1]}\ \texttt{conseq ll H}$$

The second rule is sound because a Hoare postcondition holds in every final memory of its side, and so in every pair of memories the coupling can reach. The last implication must hold in all such pairs, which is why EasyCrypt states it with the variables in $\mathrm{mod}(c_1)$ and $\mathrm{mod}(c_2)$ universally quantified.

01_conseq.ec

```
(* conseq: the shapes worth knowing.
 * Run: easycrypt compile 01_conseq.ec *)
require import AllCore.

module M = {
  var x, y : int

  proc incr() : unit = { x <- x + 1; }
  proc both() : unit = { x <- x + 1; y <- y + 1; }
  proc f(a : int) : int = { x <- x + 1; return a; }
}.

(* ---------------------------------------------------------------------- *)
(* 1. Apply a spec lemma, weakening its post / strengthening its pre.
 *    The side goals are  pre ⇒ pre'  and  post' ⇒ post. *)
lemma incr_spec (n : int) : hoare [M.incr : M.x = n ==> M.x = n + 1].
proof. by proc; auto. qed.

lemma use_spec : hoare [M.incr : 0 <= M.x ==> 0 < M.x].
proof.
  exists* M.x; elim* => n.        (* name the initial value; see 03_exlim_seq *)
  by conseq (incr_spec n) => /#.
qed.

(* ---------------------------------------------------------------------- *)
(* 2. Change the post to what the next tactic proves, keep the pre ( _ ).
 *    Typical before sim, which wants equalities. *)
lemma to_sim : equiv [M.both ~ M.both : ={M.x, M.y} ==> M.x{1} = M.x{2}].
proof.
  conseq (: _ ==> ={M.x, M.y}) => //.
  by sim.
qed.

(* Same, with the implication proved by hand instead of => // . *)
lemma to_sim' : equiv [M.both ~ M.both : ={M.x, M.y} ==> M.x{1} + M.y{1} = M.x{2} + M.y{2}].
proof.
  conseq (: _ ==> ={M.x, M.y}).
  + by move=> &1 &2 /> /#.        (* post implication, if not automatic *)
  by sim.
qed.

(* ---------------------------------------------------------------------- *)
(* 3. conseq E H1 H2 : glue one-sided hoare facts onto a two-sided equiv.
 *    An equiv relates {1} to {2}; it cannot say how a side's state relates to
 *    its OWN initial value.  Fix the initial values, then add a hoare triple
 *    per side.  Side goals: pre ⇒ P ∧ P1{1} ∧ P2{2}, and
 *    Q ∧ Q1{1} ∧ Q2{2} ⇒ post. *)
equiv f_eq : M.f ~ M.f : ={arg} ==> ={res}.
proof. by proc; auto. qed.
```

```
50
51 lemma f_x (n : int) : hoare [M.f : M.x = n ==> M.x = n + 1].
52 proof. by proc; auto. qed.
53
54 lemma glue (n1 n2 : int) :
55   equiv [M.f ~ M.f : ={arg} /\ M.x{1} = n1 /\ M.x{2} = n2
56                  ==> ={res} /\ M.x{1} = n1 + 1 /\ M.x{2} = n2 + 1].
57 proof. by conseq f_eq (f_x n1) (f_x n2). qed.
58
59 (* (_: true ==> true) = "nothing extra on this side"; it becomes a trivial goal. *)
60 lemma glue_right (n2 : int) :
61   equiv [M.f ~ M.f : ={arg} /\ M.x{2} = n2 ==> ={res} /\ M.x{2} = n2 + 1].
62 proof. by conseq f_eq (_: true ==> true) (f_x n2). qed.
63 (* Here => // closed the trivial hoare goal; otherwise: + by proc; auto. *)
64
65 (* Inside a proof, when the initial value is not a lemma argument yet:
66  *   exists* M.x{2}; elim* => n2.
67  *   conseq f_eq (_: true ==> true) (f_x n2).                          *)
68
69 (* ------------------------------------------------------------------------ *)
70 (* 4. phoare = 1 from losslessness + a hoare triple (what call{i} wants). *)
71 lemma f_ll : islossless M.f.
72 proof. by proc; auto. qed.
73
74 lemma f_ph (n : int) : phoare [M.f : M.x = n ==> M.x = n + 1] = 1%r.
75 proof. by conseq f_ll (f_x n). qed.
76
77 (* ------------------------------------------------------------------------ *)
78 (* 5. Combine two hoare triples (conjoin their posts). *)
79 lemma f_arg (a0 : int) : hoare [M.f : arg = a0 ==> res = a0].
80 proof. by proc; auto. qed.
81
82 lemma f_both (n a0 : int) :
83   hoare [M.f : M.x = n /\ arg = a0 ==> M.x = n + 1 /\ res = a0].
84 proof. by conseq (f_x n) (f_arg a0) => /#. qed.
```

4 call

- `call (: I)` handles a call to an abstract adversary with an invariant `I`. It leaves one goal per oracle procedure, of the form `={arg} /\ I ==> ={res} /\ I`. Directly on the procedure, the same thing is `proc I`.
- A procedure declared with `{}` in its module type may not call any oracle. `call (: true)` is then enough, since it only touches the adversary's own state and every other fact in the precondition carries over.
- For an abstract oracle procedure, or an alias like `proc get = O.get`, use `proc*; call (: true)`.
- `call (: I)` is only for abstract procedures. A concrete procedure needs a full spec, `call (: P ==> Q)`.
- `call{1} H` with `H` a phoare spec with probability 1 runs the call on the left side only. Its postcondition is all you learn about what the call did.
- `call H` with a hoare lemma `H` works as in hoare logic.

$$\frac{\forall \mathtt{o}.\ \models \mathtt{O}_1.\mathtt{o} \sim \mathtt{O}_2.\mathtt{o} : =\{arg\} \wedge I \Rightarrow =\{res\} \wedge I}{\models \mathtt{A}(\mathtt{O}_1).\mathtt{f} \sim \mathtt{A}(\mathtt{O}_2).\mathtt{f} : =\{arg, \mathtt{glob\ A}\} \wedge I \Rightarrow =\{res, \mathtt{glob\ A}\} \wedge I}\ \texttt{call (: I)}$$

$$\frac{\begin{array}{c}\models \mathtt{f} : P \Rightarrow Q\ [=1] \\ \Phi \Rightarrow P[e/arg]\{1\} \wedge \forall v\, \vec{w}.\ Q[v/res, \vec{w}/\mathrm{mod}(\mathtt{f})]\{1\} \Rightarrow \Psi[v/x, \vec{w}/\mathrm{mod}(\mathtt{f})]\{1\}\end{array}}{\models x \leftarrow \mathtt{f}(e) \sim \mathtt{skip} : \Phi \Rightarrow \Psi}\ \texttt{call\{1\} H}$$

In the first rule `o` ranges over the oracle procedures `A` may call, I must not mention `glob A`, and `A` must not be able to write the variables of I. A procedure declared with `{}` may call no oracle, so the rule has no premises and $I = true$ suffices, while facts about variables outside mod = `glob A` survive the call unchanged.

In the second rule the new values of everything `f` may modify are universally quantified, and Q is all that is known about them. A spec that says nothing about some variable the rest of the proof needs leaves an unprovable goal, with a fresh name such as `S_R` where the old value used to be.

02_call.ec

```
(* call: the shapes worth knowing.
 * Run: easycrypt compile O2_call.ec *)
require import AllCore.

module type Orc = {
  proc get(x : int) : int
}.

(* An adversary with one state-reading procedure that may NOT call the oracle
 * ({} = empty list of allowed oracle procedures) and one that may. *)
module type Adv (O : Orc) = {
  proc save() : int {}
  proc run() : bool
}.

module O1 : Orc = {
  var n : int
  proc get(x : int) : int = { n <- n + 1; return x; }
}.

module O2 : Orc = {
  var m : int
  proc get(x : int) : int = { m <- m + 1; return x; }
}.

(* An oracle that is a mere alias of another. *)
module W (O : Orc) : Orc = {
  proc get = O.get
}.

module G (O : Orc) (A : Adv) = {
  var s : int
  proc main() : bool = {
    var b;
    s <@ A(O).save();
    b <@ A(O).run();
    return b;
  }
}.

section.

declare module A <: Adv {-O1, -O2, -G}.
declare module O <: Orc {-A}.

(* ---------------------------------------------------------------------- *)
(* 1. Abstract adversary, different oracles: call with an invariant I.
 *    One goal per oracle procedure: ={arg} ∧ I ⟹ ={res} ∧ I.
 *    The call itself gives ={res, glob A} ∧ I from ={arg, glob A} ∧ I. *)
lemma run_rel :
  equiv [G(O1, A).main ~ G(O2, A).main :
         ={glob A} ∧ O1.n{1} = O2.m{2} ⟹ ={res} ∧ O1.n{1} = O2.m{2}].
proof.
  proc.
  call (: O1.n{1} = O2.m{2}).
  + by proc; auto.                  (* the oracle goal *)
  (* save has {} : it touches nothing but glob A, so (: true) is enough and
   * every other fact in the precondition is carried over unchanged. *)
  call (: true).
  by auto.
qed.

(* The same statement proved directly on the procedure: proc I. *)
lemma run_rel' :
  equiv [A(O1).run ~ A(O2).run :
         ={glob A} ∧ O1.n{1} = O2.m{2} ⟹ ={res, glob A} ∧ O1.n{1} = O2.m{2}].
proof.
  proc (O1.n{1} = O2.m{2}) ⇒ //.
  by proc; auto.
qed.

(* ---------------------------------------------------------------------- *)
(* 2. An abstract oracle procedure, or an alias of one: proc* then call (: true).
 *    For an abstract procedure call (: true) gives ={res, glob O} from
 *    ={arg, glob O}. *)
```

```
equiv w_get : W(O).get ~ W(O).get : ={arg, glob O} ==> ={res, glob O}.
proof. by proc*; call (: true); auto. qed.

(* ---------------------------------------------------------------------- *)
(* 3. One-sided call: call{1} / call{2} with a phoare = 1 spec.
 *    The other side does nothing; the spec's post is all you learn. *)
module K = {
  var c : int
  proc tick() : unit = { c <- c + 1; }
  proc two() : unit = { tick(); tick(); }
  proc one() : unit = { tick(); }
}.

lemma tick_ph (n : int) : phoare [K.tick : K.c = n ==> K.c = n + 1] = 1%r.
proof. by proc; auto. qed.

lemma drop_one :
  equiv [K.two ~ K.one : ={K.c} ==> K.c{1} = K.c{2} + 1].
proof.
  proc.
  (* last calls, two-sided.  For a concrete procedure give a full spec
   * (: P ==> Q); the invariant form (: I) is for abstract procedures. *)
  call (: K.c{1} = K.c{2} + 1 ==> K.c{1} = K.c{2} + 1).
  + by proc; auto.
  (* the remaining left-only call: fix K.c{1}, then call{1} *)
  exlim K.c{1} => n.
  call{1} (tick_ph n).
  by auto => /#.
qed.

(* A lossless spec drops a call whose effect you don't care about:
 *   call{1} some_ll.      where some_ll : islossless P.f
 * You lose every fact about what P.f writes. *)

(* ---------------------------------------------------------------------- *)
(* 4. call with a spec lemma instead of an invariant (hoare or equiv). *)
lemma tick_h (n : int) : hoare [K.tick : K.c = n ==> K.c = n + 1].
proof. by proc; auto. qed.

lemma two_h (n : int) : hoare [K.two : K.c = n ==> K.c = n + 2].
proof.
  proc.
  call (tick_h (n + 1)).
  call (tick_h n).
  by auto => /#.
qed.

end section.
```

5 `exlim` and `seq`

- `exlim e => v` binds `v` to the value of `e` in the precondition of the current goal. That precondition describes the state at the start of the program that is left, not at the point where you want to use `v`.
- The typical mistake is an `exlim` at the top of a proof whose program changes the value before the call that needs it. The call's precondition then asks for an equality that no longer holds.
- The fix is to cut the program first, so that the call becomes the start of what is left. `seq` cuts at any point, and `sp` consumes leading assignments.
- In a relational goal, say which side, as in `exlim x{1}, M.y{2} => a b`.
- An `exlim` at the top is fine when nothing before the call changes the value, because then the value at the start and the value at the call coincide.

$$\frac{\models c_1 \sim c_2 : \Phi \Rightarrow \Theta \qquad \models c_1' \sim c_2' : \Theta \Rightarrow \Psi}{\models c_1; c_1' \sim c_2; c_2' : \Phi \Rightarrow \Psi}\ \texttt{seq} \qquad \frac{\forall v.\ \models c_1 \sim c_2 : \Phi \wedge e = v \Rightarrow \Psi}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi}\ \texttt{exlim e => v}$$

In the second rule v is fresh, and e is evaluated in the memories Φ talks about, which are the initial memories of all of c_1 and c_2. To name a value in the middle of a program, apply `seq` first and then `exlim` in its second premise, where the middle of the program has become the start of c_1' and c_2'.

```
(* exlim / exists* and seq.
 * Run: easycrypt compile 03_exlim_seq.ec
 *
 * THE RULE: exlim e ⇒ v  binds v to the value of e in the PRECONDITION of
 * the current goal, i.e. at the START of the program that is left.  If the
 * program changes e before the point where you need v, first cut the program
 * so that point becomes the start (seq, sp), then exlim.
 *
 *   exlim e1, e2 ⇒ a b.          same as   exists* e1, e2; elim* ⇒ a b.
 *   exlim x{1}, M.y{2} ⇒ a b.    relational goals: say which side.
 *)
require import AllCore.

module P = {
  var x : int
  proc inc() : unit = { x <- x + 1; }
  proc main() : unit = { x <- 5; inc(); }
  proc main2() : unit = { inc(); }
}.

lemma inc_h (n : int) : hoare [P.inc : P.x = n ==> P.x = n + 1].
proof. by proc; auto. qed.

lemma inc_ph (n : int) : phoare [P.inc : P.x = n ==> P.x = n + 1] = 1%r.
proof. by proc; auto. qed.

(* ---------------------------------------------------------------------- *)
(* WRONG: exlim at the top binds the value BEFORE x <- 5, so the call's
 * precondition P.x = n cannot be met:
 *
 *   proc. exlim P.x ⇒ n. call (inc_h n).   (* stuck: need 5 = n *)
 *
 * RIGHT: cut first, so the call is at the start of what is left.          *)
lemma main_h : hoare [P.main : true ==> P.x = 6].
proof.
  proc.
  seq 1 : (P.x = 5); first by auto.   (* hoare seq: seq <#stmts> : (P) *)
  exlim P.x ⇒ n.                      (* now n is the value at the call *)
  call (inc_h n).
  by auto ⇒ /#.
qed.

(* For straight-line assignments at the start, sp does the cut for you. *)
lemma main_h' : hoare [P.main : true ==> P.x = 6].
proof.
  proc; sp.                           (* pre is now P.x = 5 *)
  exlim P.x ⇒ n.
  call (inc_h n).
  by auto ⇒ /#.
qed.

(* ---------------------------------------------------------------------- *)
(* Relational version: a one-sided call needs the ghost at the call point.
 * equiv seq: seq <#left> <#right> : (P).  #pre = the current precondition. *)
lemma main_eq : equiv [P.main ~ P.main2 : true ==> P.x{1} = 6].
proof.
  proc.
  seq 1 0 : (P.x{1} = 5); first by auto.
  exlim P.x{1} ⇒ n.
  call{1} (inc_ph n).
  (* right side still has its call; drop it one-sided too (no ghost needed) *)
  call{2} (: true ==> true); first by proc; auto.
  by auto ⇒ /#.
qed.

(* ---------------------------------------------------------------------- *)
(* Calls are processed from the END of the program.  exlim at the top is fine
 * when nothing before the call changes the value: then "value at the start"
 * and "value at the call" coincide. *)
lemma twice (n : int) :
  equiv [P.main2 ~ P.main2 : P.x{1} = n ∧ P.x{2} = n ==> ={P.x}].
proof.
  proc.
  exlim P.x{1} ⇒ v.                   (* fine: inc is the first statement *)
```

```
  call{1} (inc_ph v).
  exlim P.x{2} ⇒ w.
  call{2} (inc_ph w).
  by auto ⇒ /#.
qed.

(* ---------------------------------------------------------------------- *)
(* Other cutting tools
 *   seq 3 3 : (#pre ∧ ={x}).        keep the old facts and add one
 *   seq 1 1 : (Q); first by ... .    discharge the prefix inline
 *   sp / sp 1 1                      consume leading assignments into the pre
 *   wp                               consume trailing assignments into the post
 *   swap{1} 3 -2                     move left statement 3 up by 2 (needs
 *                                    syntactic independence)                 *)
```

6 Probabilities

- `rewrite Pr[lemma]` applies a distribution lemma to a probability term. The useful ones are `mu_le1`, `mu_sub`, `mu_not`, `mu_or` and `mu_split E`.
- $\Pr[E] = 0$ follows from a hoare triple on $\neg E$, by `byphoare`, then `hoare`.
- A phoare spec with probability 1 follows from a fact of the form $\Pr[\ldots] = 1$ that holds at every memory, by `bypr`.
- A hoare spec follows from the same fact. `bypr` turns the goal into $\Pr[\neg Q] = 0$, and $\Pr[\neg Q] = \Pr[\mathit{true}] - \Pr[Q]$ with $1 = \Pr[Q] \le \Pr[\mathit{true}] \le 1$. No losslessness is needed.
- Inside an event written in a proof, `X{m}` for the memory bound by `bypr` is read as the current `X`. Rather than retyping the event, rewrite with the hypothesis that already contains it, as in `rewrite -h1 Pr[mu_sub]`.
- `byequiv lemma` needs the lemma's pre- and postcondition to match exactly. State the spec in `byequiv`, discharge it with `conseq`, and use `symmetry` when the games sit on the other sides.
- `smt` treats probability terms as opaque reals, so arithmetic over several of them works as long as the terms are syntactically identical.
- $\Pr[E_1] \le \Pr[E_2]$ holds when every terminating run that ends in E_1 ends in E_2. Prove the hoare triple for $E_1 \Rightarrow E_2$, then `byequiv` with the program against itself, the triple glued on with `conseq E H1 H2`.
- $\Pr[\neg E] = 1 - \Pr[E]$ needs $\Pr[\mathit{true}] = 1$, which is termination. Where the program may not terminate, state upper bounds instead.

$$\frac{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi \qquad \Phi(m_1, m_2) \qquad \Psi \Rightarrow (E_1\{1\} \Rightarrow E_2\{2\})}{\Pr[c_1, m_1 : E_1] \le \Pr[c_2, m_2 : E_2]}\ \texttt{byequiv}$$

$$\frac{\models c_2 \sim c_1 : \Phi^{-1} \Rightarrow \Psi^{-1}}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi}\ \texttt{symmetry} \qquad \frac{\forall m.\ P(m) \Rightarrow \Pr[c, m : Q] = p}{\models c : P \Rightarrow Q\ [=p]}\ \texttt{bypr}\ \text{(phoare)}$$

$$\frac{\forall m.\ P(m) \Rightarrow \Pr[c, m : \neg Q] = 0}{\models c : P \Rightarrow Q}\ \texttt{bypr}\ \text{(hoare)} \qquad \frac{\models c : P \Rightarrow \neg E \qquad P(m)}{\Pr[c, m : E] = 0}\ \texttt{byphoare; hoare}$$

With $\Leftrightarrow$ in place of $\Rightarrow$ in the last premise, `byequiv` gives equality. Φ^{-1} is Φ with $\{1\}$ and $\{2\}$ exchanged.

04_pr.ec

```
(* Pr[...] rewriting, and moving between Pr, hoare and phoare.
 * Run: easycrypt compile 04_pr.ec *)
require import AllCore Distr DBool.

module T = {
  var x : int
  proc flip() : bool = { var b; b <$ {0,1}; return b; }
  proc keep() : unit = { }
}.

(* ---------------------------------------------------------------------- *)
(* 1. The rewrite rules: Pr[lemma] applies a distribution lemma to a Pr term. *)
lemma le1 &m : Pr[T.flip() @ &m : res] ≤ 1%r.
proof. by rewrite Pr[mu_le1]. qed.

```

```
lemma sub &m : Pr[T.flip() @ &m : res ∧ T.x = 0] ≤ Pr[T.flip() @ &m : res].
proof. by rewrite Pr[mu_sub]. qed.            (* event inclusion *)

lemma not &m :
  Pr[T.flip() @ &m : !res] = Pr[T.flip() @ &m : true] - Pr[T.flip() @ &m : res].
proof. by rewrite Pr[mu_not]. qed.

lemma or &m :
  Pr[T.flip() @ &m : res ∨ T.x = 0] =
  Pr[T.flip() @ &m : res] + Pr[T.flip() @ &m : T.x = 0]
  - Pr[T.flip() @ &m : res ∧ T.x = 0].
proof. by rewrite Pr[mu_or]. qed.

lemma split_ex &m :
  Pr[T.flip() @ &m : res] =
  Pr[T.flip() @ &m : res ∧ T.x = 0] + Pr[T.flip() @ &m : res ∧ !(T.x = 0)].
proof. by rewrite Pr[mu_split (T.x = 0)]. qed.

(* NOTE: `split` is a keyword: `lemma split` and `have split : ...` are both
 * parse errors.  Same for other tactic names.  Use hsplit, split_ex, ... *)

(* ------------------------------------------------------------------------ *)
(* 2. Pr[E] = 0 from a hoare triple on "not E". *)
lemma never &m : Pr[T.keep() @ &m : T.x ≠ T.x{m}] = 0%r.
proof. byphoare (_: T.x = T.x{m} ⟹ _) ⇒ //; hoare; proc; auto. qed.

(* ------------------------------------------------------------------------ *)
(* 3. phoare = 1 from a Pr = 1 fact that holds at every memory: bypr. *)
lemma keep_pr &m : Pr[T.keep() @ &m : T.x = T.x{m}] = 1%r.
proof. byphoare (_: T.x = T.x{m} ⟹ T.x = T.x{m}) ⇒ //; proc; auto. qed.

lemma keep_ph (n : int) : phoare [T.keep : T.x = n ⟹ T.x = n] = 1%r.
proof. by bypr ⇒ &m <-; exact (keep_pr &m). qed.

(* ------------------------------------------------------------------------ *)
(* 4. hoare from a Pr = 1 fact: bypr turns the goal into Pr[not Q] = 0.
 *    Pr[not Q] = Pr[true] - Pr[Q] = Pr[true] - 1, and 1 = Pr[Q] ≤ Pr[true] ≤ 1.
 *    No losslessness needed.
 *
 *    PITFALL: inside a Pr[...] event written in a proof, T.x{m} for the
 *    memory bound by bypr is read as the CURRENT value (the event becomes
 *    T.x = T.x).  Don't write the event by hand; get it from the hypothesis,
 *    as in  rewrite -h1 Pr[mu_sub]. *)
lemma keep_h (n : int) : hoare [T.keep : T.x = n ⟹ T.x = n].
proof.
  bypr ⇒ &m <-.
  have h1 := keep_pr &m.
  have h2 : Pr[T.keep() @ &m : true] ≤ 1%r by rewrite Pr[mu_le1].
  have h3 : 1%r ≤ Pr[T.keep() @ &m : true] by rewrite -h1 Pr[mu_sub].
  by rewrite Pr[mu_not] h1; smt().
qed.

(* ------------------------------------------------------------------------ *)
(* 5. byequiv: a lemma must match the goal's pre/post EXACTLY.  State the
 *    spec you want in byequiv, then conseq the lemma into it.  If the games
 *    are on the other sides, symmetry flips the goal ({1} and {2} swap). *)
module U = {
  proc f() : bool = { var b; b <$ {0,1}; return b; }
  proc g() : bool = { var b; b <$ {0,1}; return !b; }
}.

equiv fg : U.f ~ U.g : true ⟹ res{1} = !res{2}.
proof. by proc; auto. qed.                    (* couples b{1} = b{2} *)

lemma fg_pr &m : Pr[U.f() @ &m : res] ≤ Pr[U.g() @ &m : !res].
proof.
  byequiv (: true ⟹ res{1} = !res{2}) ⇒ //.   (* ⇒ // closed pre/post *)
  by conseq fg.
qed.

lemma gf_pr &m : Pr[U.g() @ &m : !res] ≤ Pr[U.f() @ &m : res].
proof.
  byequiv (: true ⟹ res{2} = !res{1}) ⇒ //.
  by symmetry; conseq fg ⇒ /#.
qed.
```

```
(* ---------------------------------------------------------------------- *)
(* 6. Pr arithmetic with smt: hypotheses about Pr terms are fine, smt treats
 *    the Pr terms as opaque reals.  Keep the terms syntactically identical. *)
lemma eq_from_two &m : Pr[U.f() @ &m : res] = Pr[U.g() @ &m : !res].
proof. by have := fg_pr &m; have := gf_pr &m; smt(). qed.

(* ---------------------------------------------------------------------- *)
(* 7. Pr[E1] ≤ Pr[E2] when every terminating run that ends in E1 ends in E2
 *    (a hoare fact).  byequiv with the program against itself; the hoare
 *    triple is glued on with conseq E H1 H2 (01_conseq.ec, 3.). *)
module W = {
  var a, b : bool
  proc run() : unit = { a <$ {0,1}; b <- b ∨ a; }
}.

lemma W_ab : hoare [W.run : true ⟹ W.a ⇒ W.b].
proof. by proc; auto ⇒ /#. qed.

lemma W_le &m : Pr[W.run() @ &m : W.a] ≤ Pr[W.run() @ &m : W.b].
proof.
  byequiv (: ={W.a, W.b} ⟹ ={W.a, W.b} ∧ (W.a{2} ⇒ W.b{2})) ⇒ //.
  by conseq (: ={W.a, W.b} ⟹ ={W.a, W.b}) (: true ⟹ true) W_ab ⇒ //; sim.
qed.
(* The same without byequiv: byphoare + hoare gives Pr[E1 ∧ !E2] = 0 (2.),
 * then rewrite Pr[mu_split E2] and Pr[mu_sub]. *)

(* ---------------------------------------------------------------------- *)
(* 8. Pr[!E] = 1 - Pr[E] needs Pr[true] = 1, i.e. termination: byphoare
 *    with a lossless program.  Without it, prefer stating upper bounds. *)
lemma flip_not &m : Pr[T.flip() @ &m : !res] = 1%r - Pr[T.flip() @ &m : res].
proof.
  rewrite Pr[mu_not]; congr.
  by byphoare ⇒ //; proc; auto; rewrite dbool_ll.
qed.
```

7 Up to bad

The call rule takes three components, `call (: B, I, J)`.

`B`	the bad event, read on the right memory and written without `{2}`
`I`	the relational invariant while ¬`B`, which must include the equality of the flag
`J`	what still holds once `B` is set, preserved by each side alone (optional)

It generates a goal saying that the adversary terminates when its oracles do, and then three goals for every oracle procedure `o`.

1. `equiv [o ~ o : !B{2} ∧ ={arg} ∧ I ⟹ if B{2} then J else ={res} ∧ I]`.
2. `forall &2, B{2} ⇒ phoare [o{1} : J ⟹ J] = 1%r`, for the left side after bad.
3. `forall &1, phoare [o{2} : B ∧ J ⟹ B ∧ J] = 1%r`, which says the right side keeps bad.

The code before the call is left with the postcondition `if B{2} then J else (={res, glob A} ∧ I)`.

- Choose `J` so that it implies bad stays set. If some oracle could unset bad, for instance by overwriting the record that bad is computed from, put into `J` whatever switches that oracle off.
- If you only need `!B{2} ⇒ ={res}` and not the equality of the flags afterwards, drop `J`. The goals after bad then only ask for termination.
- From the equivalence, one `byequiv` gives $\Pr_1[E] \le \Pr_2[E \vee \mathit{bad}] \le \Pr_2[E] + \Pr_2[\mathit{bad}]$.
- If $\Pr_2[\mathit{bad}] = 0$, which a hoare triple can show, a second `byequiv` in the other direction gives $\Pr_2[E \wedge \neg \mathit{bad}] \le \Pr_1[E]$. Splitting $\Pr_2[E]$ on bad then yields equality.

Writing $\mathcal{I}(X)$ for **if** $B\{2\}$ **then** J **else** $(=\{X\} \wedge I)$, the rule reads as follows.

$\mathtt{A}$ is lossless when its oracles are

$$\frac{\forall \mathtt{o}.\ \models \mathtt{O}_1.\mathtt{o} \sim \mathtt{O}_2.\mathtt{o} : \neg B\{2\} \wedge =\{arg\} \wedge I \Rightarrow \mathcal{I}(res) \qquad \forall \mathtt{o}.\ \forall m_2.\ B(m_2) \Rightarrow \left(\models \mathtt{O}_1.\mathtt{o} : J \Rightarrow J\ [=1]\right) \qquad \forall \mathtt{o}.\ \forall m_1.\ \models \mathtt{O}_2.\mathtt{o} : B \wedge J \Rightarrow B \wedge J\ [=1]}{\models \mathtt{A}(\mathtt{O}_1).\mathtt{f} \sim \mathtt{A}(\mathtt{O}_2).\mathtt{f} : \mathcal{I}(arg, \mathtt{glob\ A}) \Rightarrow \mathcal{I}(res, \mathtt{glob\ A})}\ \texttt{call (: B, I, J)}$$

$$\frac{\models c_1 \sim c_2 : \Phi \Rightarrow =\{B\} \wedge (\neg B\{2\} \Rightarrow =\{res\}) \qquad \Phi(m_1, m_2)}{\Pr[c_1, m_1 : E] \leq \Pr[c_2, m_2 : E] + \Pr[c_2, m_2 : B]}\ \text{up to bad}$$

In the third and fourth premises J is a relation, and the memory of the side that does not run is the fixed m_2 or m_1. The second rule is `byequiv` followed by $\Pr[E \vee B] \leq \Pr[E] + \Pr[B]$, where E is an event on the result.

05_upto_bad.ec

```
(* Up-to-bad: two games that agree until a flag is raised.
 * Run: easycrypt compile 05_upto_bad.ec
 *
 * The call rule has three parts:
 *
 *   call (: B, I, J).
 *     B : the bad event, read on the RIGHT memory (written without {2})
 *     I : relational invariant while not B
 *     J : what still holds once B happened (relational; optional)
 *
 * It generates, for the adversary and then for EACH oracle procedure o:
 *   - the adversary terminates when its oracles do
 *   - equiv [o ~ o : !B{2} ∧ ={arg} ∧ I ==> if B{2} then J else ={res} ∧ I]
 *   - forall &2, B{2} => phoare [o{1} : J ==> J] = 1      (left, after bad)
 *   - forall &1,         phoare [o{2} : B ∧ J ==> B ∧ J] = 1  (right keeps bad)
 * and leaves the code before the call with post
 *   if B{2} then J else (={res, glob A} ∧ I).
 *
 * Choosing I: include ={flag} itself, or nothing says the LEFT flag is
 * still down while the right one is (you then can't conclude ={Flag.bad}).
 *
 * Choosing J: it must be preserved by each side ALONE, and it must imply
 * that B stays set.  If an oracle could un-set B (e.g. overwrite the record
 * B is computed from) put in J whatever switches that oracle off.
 *)
require import AllCore Distr.

module type Orc = {
  proc f(x : int) : int
}.

module type Adv (O : Orc) = {
  proc run() : bool
}.

module Flag = {
  var bad : bool
  var s   : int
}.

(* Two oracles that differ only on the point s; both raise the flag there.
 * Nonnegative queries only, so with s < 0 the flag can never be raised. *)
module OL : Orc = {
  proc f(x : int) : int = {
    if (0 ≤ x ∧ x = Flag.s) { Flag.bad <- true; }
    return x;
  }
}.

module OR : Orc = {
  proc f(x : int) : int = {
    var r;
    r <- x;
    if (0 ≤ x ∧ x = Flag.s) { Flag.bad <- true; r <- 0; }
    return r;
  }
}.
```

```
58
59  module Game (O : Orc) (A : Adv) = {
60    proc main(s0 : int) : bool = {
61      var b;
62      Flag.bad <- false;
63      Flag.s <- s0;
64      b <@ A(O).run();
65      return b;
66    }
67  }.
68
69  section.
70
71  declare module A <: Adv {-Flag}.
72
73  declare axiom A_ll (O <: Orc {-A}) : islossless O.f ⇒ islossless A(O).run.
74
75  (* ---------------------------------------------------------------------- *)
76  (* 1. The up-to-bad equivalence. *)
77  equiv upto : Game(OL, A).main ~ Game(OR, A).main :
78    ={glob A, arg} ⟹ ={Flag.bad} ∧ (!Flag.bad{2} ⇒ ={res}).
79  proof.
80    proc.
81    call (: Flag.bad, ={Flag.s, Flag.bad}, Flag.bad{1}).
82    + exact A_ll.                              (* adversary terminates *)
83    + by proc; auto ⇒ /#.                      (* oracle, before bad *)
84    + by move⇒ &2 _; proc; auto.               (* left oracle, after bad *)
85    + by move⇒ &1; proc; auto.                 (* right oracle keeps bad *)
86    by auto ⇒ /#.
87  qed.
88
89  (* If you only need !bad{2} ⇒ ={res} (not ={Flag.bad}), drop J:
90   *   call (: Flag.bad, ={Flag.s, Flag.bad}).
91   * and the "after bad" goals only ask for termination. *)
92
93  (* ---------------------------------------------------------------------- *)
94  (* 2. The usual bound: Pr1[E] ≤ Pr2[E] + Pr2[bad]. *)
95  lemma upto_bound &m (s0 : int) :
96    Pr[Game(OL, A).main(s0) @ &m : res] ≤
97    Pr[Game(OR, A).main(s0) @ &m : res] + Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
98  proof.
99    have le : Pr[Game(OL, A).main(s0) @ &m : res] ≤
100             Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad].
101   + byequiv (: ={glob A, arg} ⟹ ={Flag.bad} ∧ (!Flag.bad{2} ⇒ ={res})) ⇒ //.
102     + by conseq upto.
103     by move⇒ &1 &2 /#.
104   have hor : Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad] ≤
105              Pr[Game(OR, A).main(s0) @ &m : res] +
106              Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
107   + by rewrite Pr[mu_or]; smt(mu_bounded).      (* drop the - Pr[res ∧ bad] *)
108   smt().
109 qed.
110
111 (* ---------------------------------------------------------------------- *)
112 (* 3. When bad provably never happens: Pr[bad] = 0 from a hoare triple on the
113  *    right game, then equality from two one-directional byequivs. *)
114 lemma no_bad &m (s0 : int) :
115   s0 < 0 ⇒ Pr[Game(OR, A).main(s0) @ &m : Flag.bad] = 0%r.
116 proof.
117   move⇒ hs; byphoare (_: arg = s0 ⟹ _) ⇒ //; hoare.
118   proc.
119   call (: !Flag.bad ∧ Flag.s < 0); first by proc; auto ⇒ /#.
120   by auto ⇒ /#.
121 qed.
122
123 lemma upto_eq &m (s0 : int) :
124   s0 < 0 ⇒
125   Pr[Game(OL, A).main(s0) @ &m : res] = Pr[Game(OR, A).main(s0) @ &m : res].
126 proof.
127   move⇒ hs; have nb := no_bad &m s0 hs.
128   (* Pr1[res] ≤ Pr2[res ∨ bad] *)
129   have le1 : Pr[Game(OL, A).main(s0) @ &m : res] ≤
130              Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad].
131   + byequiv (: ={glob A, arg} ⟹ ={Flag.bad} ∧ (!Flag.bad{2} ⇒ ={res})) ⇒ //.
132     + by conseq upto.
```

```
    by move=> &1 &2 /#.
  (* Pr2[res ∧ !bad] ≤ Pr1[res]: the games are now on the other sides *)
  have le2 : Pr[Game(OR, A).main(s0) @ &m : res ∧ !Flag.bad] ≤
             Pr[Game(OL, A).main(s0) @ &m : res].
  + byequiv (: ={glob A, arg} ==> ={Flag.bad} ∧ (!Flag.bad{1} => ={res})) => //.
    + by symmetry; conseq upto => /#.
    by move=> &1 &2 /#.
  (* arithmetic *)
  have hor : Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad] ≤
             Pr[Game(OR, A).main(s0) @ &m : res] +
             Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
  + by rewrite Pr[mu_or]; smt(mu_bounded).
  have hsplit : Pr[Game(OR, A).main(s0) @ &m : res] =
                Pr[Game(OR, A).main(s0) @ &m : res ∧ Flag.bad] +
                Pr[Game(OR, A).main(s0) @ &m : res ∧ !Flag.bad].
  + by rewrite Pr[mu_split Flag.bad].
  have hsub : Pr[Game(OR, A).main(s0) @ &m : res ∧ Flag.bad] ≤
              Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
  + by rewrite Pr[mu_sub].
  smt(mu_bounded).
qed.

end section.
```

8 Dropping one-sided calls

A rewinding reduction runs the adversary once more than the game it is compared with, and that extra run is dropped on one side only.

1. For every call you drop, get a phoare spec with probability 1 that says what it does to the state you still care about. It comes from losslessness and a hoare fact through `conseq ll H`, or from an axiom of the form $\Pr[\dots] = 1$ through `bypr`. If the axiom is existential, as a rewinding axiom usually is, these specs must be `have`s inside the proof, after `elim`.
2. Fix, with `exlim`, the values those specs need, at a point where they are right.
3. Run `call{1}` with each spec, from the last dropped call backwards.
4. Close with `auto` and `smt`.

The state-reading procedures are declared with `{}`, so they cannot call the oracle. They stand for the reduction taking a snapshot of the adversary, not for something the adversary does.

Each dropped call is one application of `call{1} H` from Section 4. For a discarded run between a save and a restore, the derivation is the following chain, read from the bottom.

$$\dfrac{\dfrac{\dfrac{\Phi \wedge \texttt{glob A}\{1\} = g \Rightarrow \Psi}{\models \texttt{load}(s) \sim \texttt{skip} : s\{1\} = f(g) \wedge \Phi \Rightarrow \Psi}\ \texttt{call\{1\} (phl g)}}{\models \texttt{run}(); \texttt{load}(s) \sim \texttt{skip} : s\{1\} = f(g) \wedge \Phi \Rightarrow \Psi}\ \texttt{call\{1\} (phr k)}}{\models s \leftarrow \texttt{save}(); \texttt{run}(); \texttt{load}(s) \sim \texttt{skip} : \texttt{glob A}\{1\} = g \wedge \Phi \Rightarrow \Psi}\ \texttt{call\{1\} (phs g)}$$

Here Φ mentions only state that `run` leaves alone, which is what the spec `phr` provides, and $\texttt{load}(f(g))$ restores `glob A` to g whatever `run` did to it.

06_one_sided.ec

```
(* Dropping one-sided calls to an abstract adversary (e.g. a discarded run in
 * a rewinding reduction).
 * Run: easycrypt compile 06_one_sided.ec
 *
 * Recipe:
 *   1. for every call you drop, a phoare [...] = 1%r spec saying what it does
 *      to the state you still care about;
 *   2. exlim the values those specs need, at a point where they are right
 *      (see 03_exlim_seq.ec);
 *   3. call{1} each spec, from the LAST dropped call backwards;
 *   4. close with auto / smt.
 *
 * Where the phoare specs come from:
```

```
 *   - losslessness + a hoare fact:  conseq ll H          (01_conseq.ec, 4.)
 *   - an axiom stated with Pr[...] = 1%r:  bypr => &m ...; exact ...
 *)
require import AllCore.

module type Orc = {
  proc get(x : int) : int
}.

(* save and load may not call the oracle ({}): they are the reduction's
 * snapshot of the adversary, not adversary actions. *)
module type Adv (O : Orc) = {
  proc save() : int {}
  proc load(s : int) : unit {}
  proc run() : bool
}.

(* An oracle that only reads its state, so a run leaves it unchanged. *)
module Cnt : Orc = {
  var k : int
  proc get(x : int) : int = { return x + k; }
}.

module G1 (A : Adv) = {
  var st : int
  proc main() : bool = {
    var b1, b2;
    st <@ A(Cnt).save();
    b1 <@ A(Cnt).run();           (* discarded run *)
    A(Cnt).load(st);
    b2 <@ A(Cnt).run();
    return b2;
  }
}.

module G2 (A : Adv) = {
  proc main() : bool = {
    var b;
    b <@ A(Cnt).run();
    return b;
  }
}.

section.

declare module A <: Adv {-Cnt, -G1}.

declare axiom A_ll (O <: Orc {-A}) : islossless O.get => islossless A(O).run.

(* Rewindability, stated with probabilities. *)
declare axiom A_rew :
  exists (f : glob A -> int),
    (forall (O <: Orc) &m,
       Pr[A(O).save() @ &m : (glob A) = (glob A){m} /\ res = f (glob A){m}] = 1%r) /\
    (forall (O <: Orc) &m (x : glob A) (s : int),
       s = f x => Pr[A(O).load(s) @ &m : (glob A) = x] = 1%r).

lemma drop_run : equiv [G1(A).main ~ G2(A).main : ={glob A, Cnt.k} ==> ={res}].
proof.
  (* 1. the specs.  f only exists after elim, so these are haves. *)
  elim A_rew => f [hs hl].
  have phs : forall g, phoare [A(Cnt).save : (glob A) = g ==> (glob A) = g /\ res = f g] = 1%r.
  + by move=> g; bypr => &m <-; exact (hs Cnt &m).
  have phl : forall g, phoare [A(Cnt).load : arg = f g ==> (glob A) = g] = 1%r.
  + by move=> g; bypr => &m hs'; exact (hl Cnt &m g arg{m} hs').
  have phr : forall k0, phoare [A(Cnt).run : Cnt.k = k0 ==> Cnt.k = k0] = 1%r.
  + move=> k0; conseq (A_ll Cnt _) (_: Cnt.k = k0 ==> Cnt.k = k0) => //.
    (* goals come out in this order: the hoare fact, then the side condition
     * of A_ll.  When unsure, look: easycrypt llm, then GOALS ALL. *)
    + by proc (Cnt.k = k0) => //; proc; auto.      (* run leaves Cnt.k *)
    by proc; auto.                                 (* Cnt.get is lossless *)
  proc.
  (* the kept run, two-sided *)
  call (: ={Cnt.k}); first by proc; auto.
  (* 2. nothing before the dropped calls changes glob A or Cnt.k, so exlim
```

```
89    *    at the top gives the right values *)
90   exlim (glob A){1}, Cnt.k{1} ⇒ g k0.
91   (* 3. backwards: load, then the run, then save *)
92   call{1} (phl g).
93   call{1} (phr k0).
94   call{1} (phs g).
95   (* 4. *)
96   by auto ⇒ /#.
97 qed.
98
99 end section.
```

9 rnd: coupling samplings

- `rnd` couples the last sampling of each side. Without arguments the two samples are equal, so the two distributions must be the same.
- `rnd f g` makes the right sample `f` of the left one, with `g` the inverse of `f`. The side goals say that `f` undoes `g` on the right support, that the masses agree, and that `f` maps the left support into the right one with `g` undoing it. For two uniform distributions on sets of the same size the mass goal is the equality of two constants.
- `rnd{1}` and `rnd{2}` take a sampling on one side only. Its distribution must be lossless, since nothing on the other side pays for its mass.
- To couple a game with itself crosswise, swap the samplings on one side so that the left `x` meets the right `y`. A symmetry argument is then one `byequiv`, because the coupling exchanges the two events. Around an adversary that cannot read the hidden values, `call (: true)` keeps the coupling.

$$\frac{\Phi \Rightarrow \big(\forall y \in d_2.\ y = f(g\,y) \wedge d_2(y) = d_1(g\,y)\big) \qquad \Phi \Rightarrow \forall x \in d_1.\ f\,x \in d_2 \wedge x = g(f\,x) \wedge \Psi[x/x\{1\},\ f\,x/y\{2\}]}{\models x \xleftarrow{\$} d_1 \sim y \xleftarrow{\$} d_2 : \Phi \Rightarrow \Psi}\ \texttt{rnd f g}$$

$$\frac{\Phi \Rightarrow \mathit{lossless}(d) \wedge \forall y \in d.\ \Psi[y/y\{2\}]}{\models \texttt{skip} \sim y \xleftarrow{\$} d : \Phi \Rightarrow \Psi}\ \texttt{rnd\{2\}}$$

$d(y)$ is the mass of y, `mu1 d y`. The bijection is only needed between the two supports, and Φ may fix the parameters it depends on, as `m{1}` does for the one-time pad.

07_rnd.ec

```
1  (* rnd: coupling the samplings of the two sides.
2   * Run: easycrypt compile 07_rnd.ec
3   *
4   * rnd works on the LAST statement of each side, so it is used after wp and
5   * before the samplings that come earlier, from the end of the program.
6   *
7   *   rnd            the two samples are equal (same distribution on both sides)
8   *   rnd f g        the right sample is f of the left one; g is the inverse of f
9   *   rnd{1}, rnd{2} a sampling on one side only (its distribution must be
10  *                  lossless: nothing on the other side pays for its mass)
11  *
12  * The side goals of rnd f g, for left x <$ dL and right y <$ dR:
13  *   forall yR, yR \in dR ⇒ yR = f (g yR)            f undoes g
14  *   forall yR, yR \in dR ⇒ mu1 dR yR = mu1 dL (g yR) masses agree
15  *   forall xL, xL \in dL ⇒ f xL \in dR ∧ xL = g (f xL) ∧ (post at xL, f xL)
16  * For two uniform distributions on sets of the same size the mass goal is
17  * the equality of the two constants.
18  *)
19 require import AllCore Bool Distr DBool DInterval IntDiv.
20
21 (* ------------------------------------------------------------------------ *)
22 (* 1. A bijection: the one-time pad.  The key is uniform, so the ciphertext
23  *    is uniform whatever the message.  f k = k ^^ m{1} sends the key to the
24  *    ciphertext and is its own inverse. *)
25 module OTP = {
26   proc enc(m : bool) : bool = { var k; k <$ {0,1}; return k ^^ m; }
```

```
  proc rand(m : bool) : bool = { var c; c <$ {0,1}; return c; }
}.

equiv otp : OTP.enc ~ OTP.rand : true ==> ={res}.
proof.
  proc; rnd (fun k => k ^^ m{1}) (fun c => c ^^ m{1}).
  by skip => /> &1; smt(xorA xorK xor_false dbool1E supp_dbool).
qed.

(* The same with a shift modulo n: f x = (x + k) %% n, g y = (y - k) %% n. *)
op n : { int | 0 < n } as gt0_n.

module Shift = {
  proc f(k : int) : int = { var x; x <$ [0..n-1]; return (x + k) %% n; }
  proc g(k : int) : int = { var y; y <$ [0..n-1]; return y; }
}.

equiv shift : Shift.f ~ Shift.g : true ==> ={res}.
proof.
  proc; rnd (fun x => (x + k{1}) %% n) (fun y => (y - k{1}) %% n).
  skip => /> &1; split => [y /supp_dinter hy | _].
  + by rewrite modzDml /= modz_small /#.
  split => [y /supp_dinter hy | _ x /supp_dinter hx].
  + by rewrite !dinter1E; smt(modz_ge0 ltz_pmod gt0_n).
  rewrite supp_dinter; split; first smt(modz_ge0 ltz_pmod gt0_n).
  by move=> _; rewrite modzDml addrK modz_small /#.
qed.

(* ---------------------------------------------------------------------- *)
(* 2. A sampling on one side only.  The right side draws a value it never
 *    uses; rnd{2} consumes it (it comes last) and asks for its losslessness. *)
module Extra = {
  proc one() : bool = { var b; b <$ {0,1}; return b; }
  proc two() : bool = { var b, u; b <$ {0,1}; u <$ [0..n-1]; return b; }
}.

equiv extra : Extra.one ~ Extra.two : true ==> ={res}.
proof.
  proc; rnd{2}; rnd; skip => /> _ _.
  by apply dinter_ll; smt(gt0_n).            (* is_lossless [0..n-1] *)
qed.

(* ---------------------------------------------------------------------- *)
(* 3. Coupling a game with ITSELF, crosswise: swap the samplings on one side
 *    so that the left x meets the right y.  rnd pairs the last samplings, so
 *    after swap{2} 1 1 it couples y{1} with x{2}, then x{1} with y{2}. *)
module Two = {
  proc pair() : int * int = {
    var x, y;
    x <$ [0..n-1];
    y <$ [0..n-1];
    return (x, y);
  }
}.

equiv mirror : Two.pair ~ Two.pair : true ==> res{1}.`1 = res{2}.`2 /\ res{1}.`2 = res{2}.`1.
proof. by proc; swap{2} 1 1; rnd; rnd; skip. qed.

(* A symmetry argument is then one byequiv: the events are exchanged. *)
lemma lt_sym &m :
  Pr[Two.pair() @ &m : res.`1 < res.`2] = Pr[Two.pair() @ &m : res.`2 < res.`1].
proof. by byequiv mirror => // /#. qed.

(* The same with an adversary in between.  Two hidden points, neither given
 * to it: its guess hits the one as often as the other.  call (: true) gives
 * ={res, glob A} because A cannot read Hidden. *)
module type Guesser = { proc guess() : int }.

module Hidden (A : Guesser) = {
  var a, b : int
  proc main() : int = {
    var g;
    a <$ [0..n-1];
    b <$ [0..n-1];
    g <@ A.guess();
```

```
    return g;
  }
}.

lemma guess_sym (A <: Guesser {-Hidden}) &m :
  Pr[Hidden(A).main() @ &m : res = Hidden.a] = Pr[Hidden(A).main() @ &m : res = Hidden.b].
proof.
  byequiv (: ={glob A} ==> ={res} /\ Hidden.a{1} = Hidden.b{2} /\ Hidden.b{1} = Hidden.a{2}) => //.
  by proc; call (: true); swap{2} 1 1; rnd; rnd; skip.
qed.
```

10 A sample the run does not read

- A game draws `x` and runs an adversary that never sees it. Inside the run nothing can say that `x` is independent of what the adversary does, because `x` is already in memory at every call.
- Move the sampling behind the call with `swap{1} 1 1; rnd; call (: true)`. The two games are then one experiment, and `byequiv` moves any event from one to the other.
- Drawn last, `x` lands in a set that the run's output determines with probability at most the mass of that set. `mu_mem_le_mu1` bounds a list of length k by k times the largest point mass.
- The phoare `seq n : R p1 p2 p3 p4` bounds the probability by $p_1p_2 + p_3p_4$. Here R is what is known of the output after the call, $p_3 = 0$ comes from a hoare triple on the call, and p_2 is the bound on the last sampling.
- This needs the run not to read `x` at all. If an oracle depends on `x`, first prove the game equivalent to one whose oracles ignore it, then move the sample.

$$\frac{\models c_1 : P \Rightarrow R\ [\le p_1] \qquad \models c_2 : R \Rightarrow Q\ [\le p_2] \qquad \models c_1 : P \Rightarrow \neg R\ [\le p_3] \qquad \models c_2 : \neg R \Rightarrow Q\ [\le p_4]}{\models c_1; c_2 : P \Rightarrow Q\ [\le p_1p_2 + p_3p_4]}\ \texttt{seq n : R p1 p2 p3 p4}$$

$$\frac{\texttt{A} \text{ cannot read or write } x}{\models x \xleftarrow{\$} d;\ v \leftarrow \texttt{A.run}() \sim v \leftarrow \texttt{A.run}();\ x \xleftarrow{\$} d : =\{\texttt{glob A}\} \Rightarrow =\{x, v, \texttt{glob A}\}}\ \texttt{swap}$$

The second rule is `swap`, then `rnd` and `call (: true)`. With $p_1 = 1$, $p_3 = 0$ and $p_4 = 1$ the first rule is the one used in `08_late_sample.ec`. With `=` in place of $\le$ everywhere it gives exact probabilities (Section 12).

`08_late_sample.ec`

```
(* A sample the run does not use: draw it after the run, then bound the
 * chance that the run's output guesses it or hits it.
 * Run: easycrypt compile 08_late_sample.ec
 *
 * A game draws x and then runs an adversary that never sees x.  Inside the
 * run nothing can say "x is independent of what the adversary does": x is
 * already in memory at every call.  Move the sampling AFTER the call
 * (swap, then rnd and call (: true)); then x is drawn when the run's output
 * is fixed, and any event "x lies in a set the output determines" has
 * probability at most the mass of that set.
 *
 *   x <$ d; v <@ A.run();    ~    v <@ A.run(); x <$ d;
 *
 * The phoare seq after the swap.  seq n : R p1 p2 p3 p4 cuts after n
 * statements and bounds the probability by p1 * p2 + p3 * p4, where
 *   p1 bounds  Pr[first part ends in R]       p2 bounds  Pr[rest hits | R]
 *   p3 bounds  Pr[first part ends in !R]      p4 bounds  Pr[rest hits | !R]
 * Here: seq 1 : (I v) 1%r p 0%r 1%r.  After the call I v holds (p3 = 0 is a
 * hoare triple on the call), and given I v the last sampling hits with
 * probability at most p.  No termination is needed: these are upper bounds.
 *)
require import AllCore List Distr DInterval.
(*---*) import StdOrder.RealOrder.

op n : { int | 0 < n } as gt0_n.

lemma mu1_d (x : int) : mu1 [0..n-1] x <= inv n%r.
```

```
proof. by rewrite dinter1E; smt(gt0_n invr_ge0). qed.

(* The adversary returns a list of at most k guesses. *)
op k : { int | 0 ≤ k } as ge0_k.

module type Adv = { proc run() : int list }.

(* The hidden point is a global of the game, so the event can mention it,
 * and A, declared {-Early}, cannot read it. *)
module Early (A : Adv) = {
  var x : int
  proc main() : int list = {
    var l;
    x <$ [0..n-1];
    l <@ A.run();
    return l;
  }
}.

module Late (A : Adv) = {
  var x : int
  proc main() : int list = {
    var l;
    l <@ A.run();
    x <$ [0..n-1];
    return l;
  }
}.

section.

declare module A <: Adv {-Early, -Late}.

declare axiom A_size : hoare[A.run : true ⟹ size res ≤ k].

(* ---------------------------------------------------------------------- *)
(* 1. The two games are one experiment: swap the sampling behind the call. *)
equiv early_late : Early(A).main ~ Late(A).main :
  ={glob A} ⟹ ={res, glob A} ∧ Early.x{1} = Late.x{2}.
proof. by proc; swap{1} 1 1; rnd; call (: true); auto. qed.

(* ---------------------------------------------------------------------- *)
(* 2. The bound, sample last.  After the call the list has at most k
 *    elements (A_size); then x falls in it with probability at most
 *    size l * (1/n) (mu_mem_le_mu1). *)
lemma late_hit &m : Pr[Late(A).main() @ &m : Late.x \in res] ≤ k%r / n%r.
proof.
  (* newer EasyCrypt makes rnd ask 0 ≤ bound: have it, ⇒ // uses it *)
  have hb : 0%r ≤ k%r / n%r by smt(ge0_k gt0_n divr_ge0).
  byphoare ⇒ //; proc.
  seq 1 : (size l ≤ k) 1%r (k%r / n%r) 0%r 1%r ⇒ //.
  + rnd (mem l) ⇒ //; skip ⇒ /> &hr hl.
    apply (ler_trans ((size l{hr})%r * inv n%r)); first exact (mu_mem_le_mu1 _ _ _ mu1_d).
    by apply ler_wpmul2r; [smt(invr_ge0 gt0_n) | rewrite le_fromint].
  by hoare; call A_size.
qed.

(* 3. ...hence with the sample first: rewrite with the equivalence. *)
lemma early_hit &m : Pr[Early(A).main() @ &m : Early.x \in res] ≤ k%r / n%r.
proof.
  have → : Pr[Early(A).main() @ &m : Early.x \in res] = Pr[Late(A).main() @ &m : Late.x \in res].
  + by byequiv early_late ⇒ // /#.
  exact (late_hit &m).
qed.

(* ---------------------------------------------------------------------- *)
(* 4. A guess: the first element of the list equals x.  No hypothesis on the
 *    adversary at all, so the seq invariant is just true. *)
lemma early_guess &m : Pr[Early(A).main() @ &m : Early.x = head 0 res] ≤ inv n%r.
proof.
  have → : Pr[Early(A).main() @ &m : Early.x = head 0 res]
         = Pr[Late(A).main() @ &m : Late.x = head 0 res].
  + by byequiv early_late ⇒ // /#.
  have hb : 0%r ≤ inv n%r by smt(gt0_n invr_ge0).
  byphoare ⇒ //; proc.
```

```
  (* with the invariant true, ⇒ // also closes the branch where it fails *)
  seq 1 : true 1%r (inv n%r) 0%r 1%r ⇒ //.
  by rnd (pred1 (head 0 l)) ⇒ //; skip ⇒ /> &hr; exact (mu1_d (head 0 l{hr})).
qed.

end section.

(* What this cannot do: bound an event that the run itself makes depend on
 * x (e.g. an oracle that answers differently at x).  First show the game
 * equivalent to one where the run does not read x at all (an equiv whose
 * oracles ignore x), then move the sample. *)
```

11 A probability as a spec, and averaging

- `Pr[G.main(x0) @ &m : res]` depends on `&m` only through `glob G`. As a phoare with precondition `arg = x0 ∧ (glob G) = (glob G){m}` it can be called inside a larger program. `bypr` reduces it to a probability from any memory with the same state, and `byequiv` with `proc true` proves that one.
- `proc true` proves an abstract procedure equivalent to itself: equal state and arguments give equal results.
- A bound for every value of a parameter is a bound for a random one. After the sampling, `exlim` names the value, `case @[ambient]` splits on a formula with no program variable in it, and the call uses the spec at that value through `conseq (: _ ⟹ _ : = p)`.
- From random to fixed: over a finite support some value is no worse than the average. A theorem about a random key, world or oracle then gives one fixed value. That value is chosen after the adversary and the memory.

$$\frac{\texttt{G abstract}}{\models \texttt{G.p} \sim \texttt{G.p} : =\{arg, \texttt{glob G}\} \Rightarrow =\{res, \texttt{glob G}\}}\ \texttt{proc true} \qquad \frac{\forall x \in d.\ \Pr[\texttt{G.main}(x), m : E] \le \varepsilon}{\Pr[x \xleftarrow{\$} d;\ \texttt{G.main}(x), m : E] \le \varepsilon}\ \text{averaging}$$

$$\frac{supp(d) \text{ finite and nonempty}}{\exists x \in d.\ \Pr[\texttt{G.main}(x), m : E] \le \Pr[x \xleftarrow{\$} d;\ \texttt{G.main}(x), m : E]}\ \text{below the average}$$

09_averaging.ec

```
(* A probability as a phoare spec, and a random parameter as an average.
 * Run: easycrypt compile 09_averaging.ec
 *
 * Pr[G.main(x0) @ &m : res] depends on &m only through glob G.  Stated as
 *   phoare[G.main : arg = x0 ∧ (glob G) = (glob G){m} ⟹ res]
 *     = (Pr[G.main(x0) @ &m : res])
 * it can be CALLED inside a bigger program: this is how a bound proved for
 * every fixed value becomes a bound for a value drawn at random, and how
 * "on average" becomes "there is a value".
 *
 * proc true proves G ~ G for an abstract G: equal state, equal arguments,
 * equal results.
 *)
require import AllCore List Distr DInterval.
(*---*) import StdOrder.RealOrder.

op n : { int | 0 < n } as gt0_n.

module type Game = { proc main(x : int) : bool }.

(* draw the parameter, then play *)
module Avg (G : Game) = {
  proc main() : bool = {
    var x, b;
    x <$ [0..n-1];
    b <@ G.main(x);
    return b;
  }
}.

(* ---------------------------------------------------------------------- *)
(* 1. A Pr term as a phoare spec: bypr, then byequiv with G ~ G. *)
```

```
lemma game_ph (G <: Game) (x0 : int) &m :
  phoare[G.main : arg = x0 ∧ (glob G) = (glob G){m} ⟹ res]
    = (Pr[G.main(x0) @ &m : res]).
proof.
  bypr ⇒ &m0.
  (* newer EasyCrypt adds 0 ≤ Pr[...] ∧ in front of the goal *)
  try (split; first by rewrite Pr[mu_ge0]).
  move⇒ -[→ hG].
  by byequiv (: ={arg, glob G} ⟹ ={res}) ⇒ //; proc true.
qed.

(* ------------------------------------------------------------------------ *)
(* 2. A bound for every value is a bound for a random one.  After the draw
 *    x is in the support; exlim names it, and the call uses game_ph at that
 *    value.  case @[ambient] splits on a formula that mentions no program
 *    variable (here xx \in [0..n-1]). *)
lemma avg_le (G <: Game) (eps : real) &m :
  (forall x, x \in [0..n-1] ⇒ Pr[G.main(x) @ &m : res] ≤ eps) ⇒
  Pr[Avg(G).main() @ &m : res] ≤ eps.
proof.
  move⇒ h.
  (* newer EasyCrypt asks 0 ≤ eps of phoare bounds: a probability is below it *)
  have heps : 0%r ≤ eps.
  + have h0 := h 0 _; first by rewrite supp_dinter; smt(gt0_n).
    have h1 : 0%r ≤ Pr[G.main(0) @ &m : res] by rewrite Pr[mu_ge0].
    smt().
  byphoare (: (glob G) = (glob G){m} ⟹ res) ⇒ //; proc.
  seq 1 : (x \in [0..n-1] ∧ (glob G) = (glob G){m}) 1%r eps 0%r 1%r ⇒ //.
  + exlim x ⇒ xx; case @[ambient]: (xx \in [0..n-1]) ⇒ hxx; last by exfalso ⇒ /#.
    call (: arg = xx ∧ (glob G) = (glob G){m} ⟹ res); last by skip ⇒ />.
    by conseq (: _ ⟹ _ : = (Pr[G.main(xx) @ &m : res])); [smt() | exact (game_ph G xx &m)].
  by hoare; auto ⇒ />; smt(supp_dinter).
qed.

(* The lower bound, the same way.  The draw now has to be lossless.  (A
 * negative eps is trivial; newer EasyCrypt wants 0 ≤ eps of the bound.) *)
lemma avg_ge (G <: Game) (eps : real) &m :
  0%r ≤ eps ⇒
  (forall x, x \in [0..n-1] ⇒ eps ≤ Pr[G.main(x) @ &m : res]) ⇒
  eps ≤ Pr[Avg(G).main() @ &m : res].
proof.
  move⇒ heps h; byphoare (: (glob G) = (glob G){m} ⟹ res) ⇒ //; proc.
  seq 1 : (x \in [0..n-1] ∧ (glob G) = (glob G){m}) 1%r eps 0%r 0%r ⇒ //.
  + rnd; skip ⇒ />; rewrite -(mu_eq_support _ predT) ?weight_dinter //; smt(gt0_n).
  exlim x ⇒ xx; case @[ambient]: (xx \in [0..n-1]) ⇒ hxx; last by exfalso ⇒ /#.
  call (: arg = xx ∧ (glob G) = (glob G){m} ⟹ res); last by skip ⇒ />.
  by conseq (: _ ⟹ _ : = (Pr[G.main(xx) @ &m : res])); [smt() | exact (game_ph G xx &m)].
qed.

(* ------------------------------------------------------------------------ *)
(* 3. From random to fixed: a value no better than the average.  Take the
 *    value that minimises the probability over the (finite) support; every
 *    value is at least as good, so avg_ge gives the average. *)
lemma argmin_list ['a] (f : 'a → real) (s : 'a list) :
  s ≠ [] ⇒ exists x, x \in s ∧ forall y, y \in s ⇒ f x ≤ f y.
proof.
  elim: s ⇒ // x s ih _; case: (s = []) ⇒ [→|hs]; first by exists x.
  by have [x0 [h0 hmin]] := ih hs; case: (f x ≤ f x0) ⇒ hx; [exists x | exists x0]; smt().
qed.

lemma exists_le_avg (G <: Game) &m :
  exists x, x \in [0..n-1] ∧ Pr[G.main(x) @ &m : res] ≤ Pr[Avg(G).main() @ &m : res].
proof.
  have hs : range 0 n ≠ [] by rewrite -size_eq0 size_range; smt(gt0_n).
  have /= [x0 [hin hmin]] := argmin_list (fun x ⇒ Pr[G.main(x) @ &m : res]) _ hs.
  exists x0; split; first by move: hin; rewrite supp_dinter mem_range /#.
  apply (avg_ge G (Pr[G.main(x0) @ &m : res]) &m); first by rewrite Pr[mu_ge0].
  by move⇒ x; rewrite supp_dinter ⇒ hx; apply hmin; rewrite mem_range /#.
qed.

(* Typical use: a theorem about a game on a random key, world or oracle
 *   Pr[Avg(G).main() @ &m : res] ≤ eps
 * gives one fixed value that the bound holds for:
 *   exists x, x \in [0..n-1] ∧ Pr[G.main(x) @ &m : res] ≤ eps
 * The value may depend on G and &m: it is chosen after them. *)
```

```
lemma fixed_value (G <: Game) (eps : real) &m :
  Pr[Avg(G).main() @ &m : res] ≤ eps ⇒
  exists x, x \in [0..n-1] ∧ Pr[G.main(x) @ &m : res] ≤ eps.
proof.
  move⇒ h; have [x [hx hle]] := exists_le_avg G &m.
  by exists x; split ⇒ //; apply (ler_trans _ _ _ hle h).
qed.
```

12 Laws, and equivalences `rnd` cannot prove

- The law of "draw coins, then run a procedure" is an exact phoare when the procedure is known by a hoare spec and terminates. Use `seq 1 : R p1 1%r _ 0%r` with `R` saying that the coins already decide the event. From `R` the call ends in the event with probability 1, by the spec with losslessness (`conseq ll H`), and from $\neg$`R` with probability 0, by a hoare triple. The `_` lets EasyCrypt fill in p_3.
- `byphoare` with that law turns the probability of an event of the program into a probability over the coins, which is a fact about distributions.
- `rnd` needs a bijection between the two samplings. When there is none, as between a pair and one value, `bypr (res{1}) (res{2})` reduces the equivalence to the equality of the two output laws, with one `byphoare` per side.
- `transitivity{1} { s }` replaces the whole left program by the statement `s`, typically a call of a procedure that an equivalence is already known about. Cut with `seq` first, so that the statements to replace are the whole program.
- `transitivity N.p (P1 ==> Q1) (P2 ==> Q2)` goes through a third procedure. Its side goals ask for a middle argument and for the composition of the two postconditions.

$$\frac{\forall m_1\, m_2\, v.\ \Phi(m_1,m_2) \Rightarrow \Pr[c_1, m_1 : res = v] = \Pr[c_2, m_2 : res = v]}{\models c_1 \sim c_2 : \Phi \Rightarrow =\{res\}}\ \texttt{bypr (res\{1\}) (res\{2\})}$$

$$\frac{\begin{array}{c}\models c_1 \sim c : \Phi_1 \Rightarrow \Psi_1 \qquad \models c \sim c_2 : \Phi_2 \Rightarrow \Psi_2 \\ \Phi(m_1,m_2) \Rightarrow \exists m.\ \Phi_1(m_1,m) \wedge \Phi_2(m,m_2) \\ \Psi_1(m_1,m) \wedge \Psi_2(m,m_2) \Rightarrow \Psi(m_1,m_2)\end{array}}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi}\ \texttt{transitivity}$$

The first rule is sound because two subdistributions that give every value the same mass are equal, and equal distributions are coupled by the identity.

10_laws.ec

```
(* Exact laws, and equivalences that rnd cannot prove.
 * Run: easycrypt compile 10_laws.ec
 *
 * - The law of "draw coins, then run a deterministic procedure" is an exact
 *   phoare, from a hoare spec of the procedure and its losslessness.
 * - Two programs whose results have the same law are equivalent, even when
 *   no bijection relates their samplings: bypr (res{1}) (res{2}), then one
 *   Pr per side.
 * - transitivity chains equivalences, and transitivity{1} { s } replaces a
 *   statement by another one (e.g. a sampling by a procedure call) so that a
 *   lemma about procedures can be used with call.
 *)
require import AllCore List Distr DInterval.

op n : { int | 0 < n } as gt0_n.

lemma dn_ll : is_lossless [0..n-1].
proof. by apply dinter_ll; smt(gt0_n). qed.

(* ------------------------------------------------------------------------ *)
(* 1. The law of a program that draws coins and runs a procedure.  All that
 *    is known of D is that it terminates and computes F, leaving its state
 *    as it was. *)
op F : int → int.

module type Det = { proc run(r : int) : int }.
```

```
27
28  module S (D : Det) = {
29    proc f() : int = {
30      var r, y;
31      r <$ [0..n-1];
32      y <@ D.run(r);
33      return y;
34    }
35  }.
36
37  section.
38
39  declare module D <: Det.
40
41  declare axiom D_ll : islossless D.run.
42
43  declare axiom D_spec (r0 : int) (g0 : glob D) :
44    hoare[D.run : r = r0 ∧ (glob D) = g0 ==> res = F r0 ∧ (glob D) = g0].
45
46  (* the spec with termination: the form a call in a phoare takes *)
47  lemma D_pr (r0 : int) (g0 : glob D) :
48    phoare[D.run : r = r0 ∧ (glob D) = g0 ==> res = F r0 ∧ (glob D) = g0] = 1%r.
49  proof. by conseq D_ll (D_spec r0 g0). qed.
50
51  (* phoare seq with five numbers, for an EXACT bound:
52   *   seq 1 : R p1 p2 p3 p4      Pr = p1 * p2 + p3 * p4
53   * R is "the coins already decide the event"; p2 = 1 and p4 = 0 because
54   * the procedure computes F (the p4 branch is a hoare triple).  The _ for p3
55   * is filled in from p1. *)
56  lemma S_law (P : int → bool) :
57    phoare[S(D).f : true ==> P res] = (mu [0..n-1] (fun r ⇒ P (F r))).
58  proof.
59    proc.
60    seq 1 : (P (F r)) (mu [0..n-1] (fun r ⇒ P (F r))) 1%r _ 0%r ⇒ //.
61    + by rnd; skip.
62    + by exists* r, (glob D); elim* ⇒ r0 g0; call (D_pr r0 g0); skip ⇒ />.
63    by hoare; exists* r, (glob D); elim* ⇒ r0 g0; call (D_spec r0 g0); skip ⇒ />.
64  qed.
65
66  (* The probability of an event of the program is then a probability over
67   * the coins, which is a fact about distributions. *)
68  lemma S_pr &m : Pr[S(D).f() @ &m : res = 0] = mu [0..n-1] (fun r ⇒ F r = 0).
69  proof. by byphoare (S_law (pred1 0)). qed.
70
71  end section.
72
73  (* ------------------------------------------------------------------------ *)
74  (* 2. Same law, no bijection.  The left side draws a PAIR and keeps its first
75   *    component; the right side draws one value.  rnd would need a bijection
76   *    between pairs and values.  bypr reduces the equivalence to the equality
77   *    of the two laws, value by value. *)
78  module Coins = {
79    proc pair() : int = { var p; p <$ [0..n-1] `*` [0..n-1]; return p.`1; }
80    proc one() : int = { var x; x <$ [0..n-1]; return x; }
81  }.
82
83  equiv pair_one : Coins.pair ~ Coins.one : true ==> ={res}.
84  proof.
85    (* the goal: forall &1 &2 v, pre ⇒ Pr[left : res = v] = Pr[right : res = v];
86     * here the pre is true and ⇒ // removes it *)
87    bypr (res{1}) (res{2}) ⇒ // &1 &2 v.
88    have → : Pr[Coins.pair() @ &1 : res = v]
89             = mu ([0..n-1] `*` [0..n-1]) (fun (p : int * int) ⇒ p.`1 = v).
90    + by byphoare ⇒ //; proc; rnd; skip.
91    have → : Pr[Coins.one() @ &2 : res = v] = mu [0..n-1] (pred1 v).
92    + by byphoare ⇒ //; proc; rnd; skip.
93    by rewrite (dprodEl _ _ (pred1 v)) dn_ll.
94  qed.
95
96  (* ------------------------------------------------------------------------ *)
97  (* 3. transitivity{1}: a sampling statement where a lemma about a procedure
98   *    is wanted.  Replace the left statement by a call of Coins.pair (first
99   *    premise: inline), then call pair_one (second premise).  The two
100  *    relations are the pre/post of the two halves; the side goals say they
101  *    compose. *)
```

```
module G1 = {
  proc main() : int = {
    var p, x;
    p <$ [0..n-1] `*` [0..n-1];
    x <- p.`1;
    return x + 1;
  }
}.

module G2 = {
  proc main() : int = {
    var x;
    x <@ Coins.one();
    return x + 1;
  }
}.

equiv g1_g2 : G1.main ~ G2.main : true ==> ={res}.
proof.
  proc.
  (* cut first: the statements to replace must be ALL of the left program *)
  seq 2 1 : (={x}); last by auto.
  transitivity{1} { x <@ Coins.pair(); } (true ==> ={x}) (true ==> ={x}) => //.
  + by inline *; auto.
  by call pair_one.
qed.

(* transitivity at the level of procedures, through a third program.  Its
 * side goals ask for a middle argument for every related pair, and that the
 * two posts compose; => // closes both here. *)
module Gp = {
  proc main() : int = {
    var x;
    x <@ Coins.pair();
    return x + 1;
  }
}.

equiv g1_g2' : G1.main ~ G2.main : true ==> ={res}.
proof.
  transitivity Gp.main (true ==> ={res}) (true ==> ={res}) => //.
  + by proc; inline *; auto.
  by proc; call pair_one.
qed.
```

13 Loops

- `while (I)` in hoare logic: `I` holds before the loop and the body keeps it under the guard; after the loop `I` holds and the guard is false.
- `while (I) (v)` for termination: from the guard, `I` and `v = z`, the body ends in `I` with `v < z`, with probability 1. Before the loop, `v ≤ 0` must imply that the guard is false.
- A loop is best given a functional spec, a hoare triple `res =` the function of the arguments, together with losslessness. If it borrows a global variable, the spec says that it puts it back, with the initial value fixed by a lemma parameter.
- Two loops that do not run in step are equivalent because they compute the same function. `proc*` turns the procedures into call statements, `exists*` and `elim*` name the arguments, and `call{1}` and `call{2}` run each side alone with its spec made a phoare with probability 1.
- `ecall (spec e)` applies a spec whose arguments are program expressions, read just before the call. In a loop body that saves an `exists*`.
- `rcondt n` and `rcondf n` decide the `if` or `while` at statement `n`, given what the first $n-1$ statements establish.

$$\frac{\models c : I \wedge b \Rightarrow I}{\models \textbf{while}\ b\ \textbf{do}\ c : I \Rightarrow I \wedge \neg b}\ \texttt{while I}$$

$$\frac{\forall z.\ \models c : I \wedge b \wedge v = z \Rightarrow I \wedge v < z\ [=1] \qquad I \wedge v \leq 0 \Rightarrow \neg b}{\models \textbf{while}\ b\ \textbf{do}\ c : I \Rightarrow I \wedge \neg b\ [=1]}\ \texttt{while I v}$$

```
(* Loops: invariants, variants, and replacing a loop by the function it
 * computes.
 * Run: easycrypt compile 11_while.ec
 *
 *   while (I)          hoare: I holds before, and the body keeps I while the
 *                      guard holds; after the loop: I ∧ !guard
 *   while (I) (v)      losslessness (phoare = 1): for every z, the body run
 *                      from guard ∧ I ∧ v = z terminates with I ∧ v < z;
 *                      and before the loop, v ≤ 0 must imply !guard
 *   rcondt n / rcondf n   the if (or while) at statement n is taken / not
 *                      taken: prove its condition after the first n-1
 *                      statements
 *
 * A procedure that loops is best given a functional spec, a hoare triple
 * "res = the function of the arguments", plus termination.  The spec is
 * then used on each side of an equiv separately (call{1}, call{2}), which
 * works even when the two loops do not run in step.
 *)
require import AllCore List.

op f : int → int.

module M = {
  var tmp : int                       (* a global the procedures borrow *)

  (* map f, left to right *)
  proc fwd(l : int list) : int list = {
    var i, r, s;
    s <- tmp;
    i <- 0; r <- [];
    while (i < size l) {
      tmp <- f (nth 0 l i);
      r <- rcons r tmp;
      i <- i + 1;
    }
    tmp <- s;
    return r;
  }

  (* map f, right to left *)
  proc bwd(l : int list) : int list = {
    var i, r;
    i <- size l; r <- [];
    while (0 < i) {
      i <- i - 1;
      r <- f (nth 0 l i) :: r;
    }
    return r;
  }
}.

(* ---------------------------------------------------------------------- *)
(* 1. A hoare invariant.  r = map f (take i l), and take (i+1) l is
 *    rcons (take i l) (nth 0 l i) (take_nth).  The global the loop borrows
 *    is put back: the spec says so, with an initial value t0. *)
lemma fwd_spec (l0 : int list) (t0 : int) :
  hoare[M.fwd : l = l0 ∧ M.tmp = t0 ⟹ res = map f l0 ∧ M.tmp = t0].
proof.
  proc; wp.
  while (0 ≤ i ≤ size l ∧ r = map f (take i l) ∧ s = t0).
  + by auto ⇒ /> &hr h0 _ hlt; rewrite (take_nth 0) // map_rcons /#.
  by auto ⇒ />; smt(take0 take_size size_ge0).
qed.

(* 2. The variant: size l - i.  The goal has a fresh z for its value. *)
lemma fwd_ll : islossless M.fwd.
proof.
  proc; wp; while (true) (size l - i).
  + by move⇒ z; auto ⇒ /#.
  by auto ⇒ /#.
qed.

lemma bwd_spec (l0 : int list) :
  hoare[M.bwd : l = l0 ⟹ res = map f l0].
```

```
proof.
  proc.
  while (0 ≤ i ≤ size l ∧ r = map f (drop i l) ∧ l = l0).
  + auto ⇒ /> &hr h0 hle hlt; rewrite (drop_nth 0 (i{hr} - 1) l0) 1:/# /= /#.
  by auto ⇒ />; smt(drop0 drop_size size_ge0).
qed.

lemma bwd_ll : islossless M.bwd.
proof.
  by proc; while (true) (i); auto ⇒ /#.
qed.

(* ---------------------------------------------------------------------- *)
(* 3. Two loops that do not run in step are equivalent because they compute
 *    the same function.  Name the arguments and the borrowed global with
 *    exists* and elim*, then run each side alone with its spec made a
 *    phoare = 1 (conseq ll spec). *)
equiv fwd_bwd : M.fwd ~ M.bwd : ={l} ⟹ ={res}.
proof.
  proc*.                              (* the calls become statements: r <@ M.fwd(l) *)
  exists* l{1}, M.tmp{1}; elim* ⇒ l0 t0.
  call{1} (: l = l0 ∧ M.tmp = t0 ⟹ res = map f l0 ∧ M.tmp = t0).
  + by conseq fwd_ll (fwd_spec l0 t0).
  call{2} (: l = l0 ⟹ res = map f l0).
  + by conseq bwd_ll (bwd_spec l0).
  by skip ⇒ />.
qed.

(* ---------------------------------------------------------------------- *)
(* 4. A loop that calls an oracle with a spec: ecall (lemma args) applies the
 *    spec at the current values of program variables, which a plain call
 *    cannot do from inside the loop body. *)
op g : int → int.

module type Orc = { proc get(x : int) : int }.

module Loop (O : Orc) = {
  proc run(l : int list) : int list = {
    var i, r, y;
    i <- 0; r <- [];
    while (i < size l) {
      y <@ O.get(nth 0 l i);
      r <- rcons r y;
      i <- i + 1;
    }
    return r;
  }
}.

section.

declare module O <: Orc.

declare axiom O_spec (x0 : int) : hoare[O.get : x = x0 ⟹ res = g x0].

lemma run_spec (l0 : int list) : hoare[Loop(O).run : l = l0 ⟹ res = map g l0].
proof.
  proc.
  while (0 ≤ i ≤ size l ∧ r = map g (take i l) ∧ l = l0).
  + wp; ecall (O_spec (nth 0 l i)); auto ⇒ /> &hr h0 _ hlt.
    by rewrite (take_nth 0) // map_rcons /#.
  by auto ⇒ />; smt(take0 take_size size_ge0).
qed.

end section.

(* ---------------------------------------------------------------------- *)
(* 5. A branch whose condition is known.  rcondt 2 says the if at statement 2
 *    is taken (after statement 1, c < 1), and leaves its body inline. *)
module Once = {
  var c : int
  proc f() : int = {
    var r;
    r <- 0;
    c <- 0;
```

```
    if (c < 1) { c <- c + 1; r <- 5; }
    return r;
  }
}.

lemma once : hoare[Once.f : true ==> res = 5].
proof. by proc; rcondt 3; [auto | auto]. qed.
```

14 `fel`: a charge per oracle call

- An oracle that raises `bad` with probability at most δ at each of at most q calls gives $\Pr[bad] \le q\delta$, whatever the adversary does between the calls. `fel` is that union bound. It needs `require import FelTactic`.
- `fel k cnt delta q bad [o : (guard); ...] inv` takes the number k of statements before the adversary runs, the counter and its charge, the bound q on the counter, the failure event, the charged oracles with the guard under which they are charged, and an optional invariant.
- Name the oracles as the adversary calls them. An entry that names no oracle is silently ignored, and that oracle gets the guard `true`.
- The goals come in a fixed order: the sum of the charges, the event, the first k statements, and then three goals per oracle, in the alphabetical order of the oracles' module names. The three are the charge, the fact that a charged call pays one step of the counter, and the fact that an uncharged call raises nothing.

$$\frac{\begin{array}{c}\sum_{i<q}\delta(i) \le \varepsilon \qquad E \wedge I \Rightarrow F \wedge cnt \le q \qquad \models c_0 : \Phi \Rightarrow \neg F \wedge cnt = 0 \wedge I \\ \forall \mathtt{o}.\ \models \mathtt{o} : 0 \le cnt < q \wedge \neg F \wedge I \wedge g_{\mathtt{o}} \Rightarrow F\ [\le \delta(cnt)] \\ \forall \mathtt{o}\, c.\ \models \mathtt{o} : g_{\mathtt{o}} \wedge cnt = c \wedge I \Rightarrow c < cnt \wedge I \\ \forall \mathtt{o}\, b\, c.\ \models \mathtt{o} : \neg g_{\mathtt{o}} \wedge F = b \wedge cnt = c \wedge I \Rightarrow F = b \wedge c \le cnt \wedge I\end{array}}{\Pr[c_0;\ \mathtt{A}^{\vec{\mathtt{o}}}, m : E] \le \varepsilon}\ \mathtt{fel}$$

Here c_0 is the first k statements and Φ says the memory is m. The charge may depend on the counter but not on the rest of the state; for that, use an ehoare potential (Section 15).

12_fel.ec

```
(* fel: a bad event charged per oracle call, with a counter.
 * Run: easycrypt compile 12_fel.ec
 *
 * An oracle that, at each of at most q calls, raises bad with probability
 * at most 1/n gives Pr[bad] ≤ q/n, whatever the adversary does between the
 * calls.  fel is that union bound over the calls.  It needs
 * require import FelTactic.
 *
 *   fel k cnt delta q bad [o1 : (guard1); o2 : (guard2); ...] inv
 *
 *   k       the number of statements of the game BEFORE the adversary runs
 *           (they must set cnt to 0 and bad to false, and establish inv)
 *   cnt     the counter, delta its charge: the call made when cnt = c may
 *           raise bad with probability at most delta c
 *   q       the bound on the counter in the event
 *   bad     the failure event
 *   [o : guard]  when an oracle is charged.  Name the oracle as the
 *           adversary calls it (O.f, not the procedure O.f is an alias of):
 *           an entry that names no oracle is silently ignored, and that
 *           oracle gets the guard true.
 *   inv     an invariant of every oracle call (optional; true if absent)
 *
 * The goal must be Pr[G : E] ≤ bound.  The goals fel leaves, in order
 * (some are closed by ⇒ //):
 *   1. bigi predT delta 0 q ≤ bound                the sum of the charges
 *   2. forall &hr, E ⇒ inv ⇒ bad ∧ cnt ≤ q       the event
 *   3. the first k statements end in !bad ∧ cnt = 0 ∧ inv
 *   and, for each oracle, in the alphabetical order of the module names
 *   (not the order of the list):
 *   4. phoare[o : 0 ≤ cnt < q ∧ !bad ∧ inv ∧ guard ==> bad] ≤ delta cnt
 *   5. forall c, hoare[o : guard ∧ c = cnt ∧ inv ==> c < cnt ∧ inv]
 *   6. forall b c, hoare[o : !guard ∧ bad = b ∧ cnt = c ∧ inv
 *                           ==> bad = b ∧ c ≤ cnt ∧ inv]
```

```
 * Goal 4 is the charge, goal 5 says a charged call pays one step of the
 * counter, goal 6 that an uncharged call raises nothing.
 *)
require import AllCore List Distr DInterval StdBigop FelTactic.
(*---*) import StdOrder.RealOrder.

op n : { int | 0 < n } as gt0_n.
op q : { int | 0 ≤ q } as ge0_q.

module type Orc = { proc f() : int }.
module type Adv (O : Orc) = { proc run() : bool }.

module O : Orc = {
  var cnt : int
  var bad : bool

  proc f() : int = {
    var x;
    x <- 0;
    if (cnt < q) {
      cnt <- cnt + 1;
      x <$ [0..n-1];
      bad <- bad ∨ x = 0;
    }
    return x;
  }
}.

module G (A : Adv) = {
  proc main() : bool = {
    var b;
    O.cnt <- 0;
    O.bad <- false;
    b <@ A(O).run();
    return b;
  }
}.

lemma mu_hit : mu [0..n-1] (pred1 0) = inv n%r.
proof. by rewrite dinter1E; smt(gt0_n). qed.

section.

declare module A <: Adv {-O}.

lemma pr_bad &m : Pr[G(A).main() @ &m : O.bad] ≤ q%r / n%r.
proof.
  (* the event must say that the counter stays within q: carry it in inv *)
  fel 2 O.cnt (fun _ ⇒ inv n%r) q O.bad [O.f : (O.cnt < q)] (O.cnt ≤ q) ⇒ //.
  + (* 1. q charges of 1/n *)
    by rewrite (Bigreal.BRA.sumri_const (inv n%r) 0 q ge0_q) /= RField.intmulr.
  + (* 3. the first two statements *)
    by auto ⇒ />; exact ge0_q.
  + (* 4. one served call raises bad with probability at most 1/n *)
    proc; rcondt 2; first by auto.
    (* newer EasyCrypt also asks 0 ≤ 1/n here: ?mu_hit skips that goal *)
    wp; rnd (pred1 0); auto ⇒ />; rewrite ?mu_hit /= ⇒ *; smt(gt0_n invr_ge0).
  + (* 5. a served call increments the counter *)
    by move⇒ c; proc; rcondt 2; [auto | auto ⇒ /#].
  (* 6. a refused call changes nothing *)
  by move⇒ b c; proc; rcondf 2; auto.
qed.

end section.
```

15 ehoare: expectations

- `ehoare[c : f ⟹ g]` says that the expected value of `g` after `c` is at most `f` before it. Both are `xreals`: nonnegative reals or infinity. `x%xr`, `i%xr` and `b%xr` coerce a real, an integer and a Boolean (1 or 0).
- `byehoare (: f ⟹ E%xr)` proves `Pr[c @ &m : E] ≤ f{m}`, because the probability of an event is the expectation of its indicator. It gives upper bounds only.

- `(b `|` f)` is `f` where `b` holds and infinity elsewhere: a guarded precondition. `xle_cxr_r` and `xle_cxr_l` open it on either side of $\leq$. In ehoare an `if` puts its condition in front of each branch's precondition in that form.
- In ehoare, `wp` goes through samplings and turns them into `Ep`. `EpD` splits a sum, `EpC` takes a constant out with the weight, and `Ep_mu` turns an indicator into a probability. After `/=` a comparison of `xreal`s of reals is a comparison of reals over `%pos`, which `smt` can do.
- `call (: inv)` through an abstract adversary needs `ehoare[o : inv ==> inv]` for each oracle, whatever state it starts from. The whole run then does not increase `inv` in expectation, whatever the adversary does between calls. This is `fel` with a charge that may depend on the state.
- `conseq` on an ehoare goal returns the postcondition implication as a guard over the modified variables. `sp` does not apply to ehoare goals; cut with `seq 1 : (inv)` instead.
- No `conseq` combines an ehoare with a hoare spec. Prove the ehoare for a reference program that computes the function, the equiv between the program and the reference, and apply `conseq E H`.

$$\frac{\models_{\mathbb{E}} c : f \Rightarrow [E]}{\Pr[c, m : E] \leq f(m)}\ \texttt{byehoare} \qquad \frac{\forall \texttt{o}.\ \models_{\mathbb{E}} \texttt{O.o} : inv \Rightarrow inv}{\models_{\mathbb{E}} \texttt{A(O).f} : inv \Rightarrow inv}\ \texttt{call (: inv)}$$

$$\frac{\begin{array}{c}\models c \sim c' : \Phi \Rightarrow \Psi \qquad \models_{\mathbb{E}} c' : f' \Rightarrow g' \\ \forall m_1.\ f(m_1) = \infty \vee \exists m_2.\ \Phi(m_1, m_2) \wedge f'(m_2) \leq f(m_1) \\ \forall m_1\, m_2.\ \Psi(m_1, m_2) \Rightarrow g(m_1) \leq g'(m_2)\end{array}}{\models_{\mathbb{E}} c : f \Rightarrow g}\ \texttt{conseq E H}$$

$[E]$ is the indicator of E. In the second rule, as for `call (: I)` in Section 4, `inv` must not mention `glob A` and `A` must not be able to write its variables. The third rule is sound because the coupling carries the expectation of g' over to c: the expected value of g after c is at most that of g' after c'.

13_ehoare.ec

```
(* ehoare: expectations instead of events.
 * Run: easycrypt compile 13_ehoare.ec
 *
 *   ehoare[c : f ==> g]     the expected value of g after c is at most f
 *                           before it; f, g : xreal (nonnegative, maybe oo)
 *
 * The bridge to probabilities is the indicator: Pr[c : E] is the
 * expectation of E%xr.  So
 *   byehoare (: f ==> E%xr)     proves  Pr[c @ &m : E] <= f{m}
 * (an upper bound only).
 *
 * Coercions: x%xr for a real, i%xr for an int, b%xr for a bool (1 or 0).
 * (b `|` f) is f when b holds and oo otherwise: a precondition guarded by b.
 *
 * What makes ehoare worth it: call (: inv) through an abstract adversary,
 * with an invariant inv that is an expectation.  Each oracle call must not
 * increase inv on average, whatever state it starts from; then the whole
 * run does not either, whatever the adversary does between calls.  The
 * same union bound as 12_fel.ec, as one invariant.
 *)
require import AllCore List Distr DInterval Xreal.
(*---*) import StdOrder.RealOrder.

op n : { int | 0 < n } as gt0_n.
op q : { int | 0 <= q } as ge0_q.

lemma mu_hit : mu [0..n-1] (pred1 0) = inv n%r.
proof. by rewrite dinter1E; smt(gt0_n). qed.

(* ---------------------------------------------------------------------- *)
(* 1. One sampling. *)
module F = {
  proc hit() : bool = { var x; x <$ [0..n-1]; return x = 0; }
}.

ehoare hit_e : F.hit : (inv n%r)%xr ==> res%xr.
proof.
  proc; rnd; skip => &hr /=.
  by rewrite Ep_mu -/(pred1 0) mu_hit.
qed.
```

```
41
42  lemma hit_pr &m : Pr[F.hit() @ &m : res] ≤ inv n%r.
43  proof.
44    have hb : 0%r ≤ inv n%r by smt(gt0_n invr_ge0).
45    by byehoare hit_e.
46  qed.
47
48  (* ---------------------------------------------------------------------- *)
49  (* 2. The union bound of 12_fel.ec as an expectation invariant.  The
50   *    potential is bad plus what the remaining calls may still add:
51   *      pot = bad + (q - cnt) / n.
52   *    A served call turns (bad, cnt) into (bad ∨ x = 0, cnt + 1); on
53   *    average it adds at most 1/n to bad and removes exactly 1/n from the
54   *    second term. *)
55  module type Orc = { proc f() : int }.
56  module type Adv (O : Orc) = { proc run() : bool }.
57
58  module O : Orc = {
59    var cnt : int
60    var bad : bool
61
62    proc f() : int = {
63      var x;
64      x <- 0;
65      if (cnt < q) {
66        cnt <- cnt + 1;
67        x <$ [0..n-1];
68        bad <- bad ∨ x = 0;
69      }
70      return x;
71    }
72  }.
73
74  module G (A : Adv) = {
75    proc main() : bool = {
76      var b;
77      O.cnt <- 0;
78      O.bad <- false;
79      b <@ A(O).run();
80      return b;
81    }
82  }.
83
84  op pot (bad : bool) (cnt : int) : xreal = bad%xr + ((q - cnt)%r / n%r)%xr.
85
86  (* One served call does not increase pot on average.  The xreal goal turns
87   * into real arithmetic over %pos once the expectations are computed: EpD
88   * splits a sum, EpC a constant (times the weight), Ep_mu an indicator. *)
89  lemma pot_step (b : bool) (c : int) : c < q ⇒
90    Ep [0..n-1] (fun x ⇒ pot (b ∨ x = 0) (c + 1)) ≤ pot b c.
91  proof.
92    move⇒ hc; have hn : 0 ≤ n - 1 by smt(gt0_n).
93    rewrite /pot; apply (xle_trans (Ep [0..n-1] (fun x ⇒ b%xr + ((x = 0)%xr + ((q - (c + 1))%r / n%r)%xr)))).
94    + by apply le_Ep ⇒ x _ /=; rewrite /(%pos); smt().
95    rewrite EpD EpC EpD EpC Ep_mu weight_dinter hn /= -/(pred1 0) mu_hit /(%pos).
96    have → : (q - c)%r / n%r = inv n%r + (q - (c + 1))%r / n%r by field; smt(gt0_n).
97    smt(gt0_n invr_ge0 divr_ge0).
98  qed.
99
100 (* The oracle keeps the invariant.  Take x <- 0 off with seq 1: sp does not
101  * apply to ehoare goals.  In ehoare an if puts its condition in front of
102  * each branch's pre, as (cond) `|` pot: open it with xle_cxr_r. *)
103 ehoare O_pot : O.f : pot O.bad O.cnt ⟹ pot O.bad O.cnt.
104 proof.
105   proc; seq 1 : (pot O.bad O.cnt); first by auto.
106   if; last by skip ⇒ &hr; apply xle_cxr_r ⇒ _; apply xlexx.
107   (* in ehoare, wp also goes through the sampling: it becomes an Ep *)
108   wp; skip ⇒ &hr /=; apply xle_cxr_r ⇒ hlt.
109   exact (pot_step O.bad{hr} O.cnt{hr} hlt).
110 qed.
111
112 (* The whole run: call (: inv) through the adversary.  The event is below
113  * the potential (conseq), and the potential starts at q/n. *)
114 lemma pr_bad (A <: Adv {-O}) &m : Pr[G(A).main() @ &m : O.bad] ≤ q%r / n%r.
115 proof.
```

```
  (* newer EasyCrypt also asks for 0 ≤ bound: have it, ⇒ // uses it *)
  have hb : 0%r ≤ q%r / n%r by smt(ge0_q gt0_n divr_ge0).
  byehoare (: (q%r / n%r)%xr ⟹ O.bad%xr) ⇒ //.
  (* conseq in ehoare: the post implication comes back as a guard
   * (forall modified variables, new post ≤ old post) `|` ... on both sides *)
  proc; conseq (: _ ⟹ pot O.bad O.cnt).
  + move⇒ &hr; apply xle_cxr_l; last by apply xlexx.
    by move⇒ b c; rewrite /pot; apply xle_add_r.
  + by move⇒ &hr; apply xle_cxr_r ⇒ h; apply h.
  call (: pot O.bad O.cnt); first by conseq O_pot.
  by wp; skip ⇒ &hr; rewrite /pot /=; smt(ge0_q gt0_n divr_ge0).
qed.

(* ---------------------------------------------------------------------- *)
(* 3. An ehoare judgment carried across an equiv.  There is no conseq that
 *    combines an ehoare with a hoare spec, so a program that calls a
 *    procedure known only by its hoare spec is handled through a reference
 *    program that computes the function directly:
 *      ehoare for the reference (no call in it),
 *      equiv program ~ reference (where the spec is usable, one-sided),
 *      conseq E H on the program.
 *    The side goals of conseq E H: for every left memory there are values
 *    of what the right pre mentions with E's pre and H's pre ≤ ours (or
 *    ours infinite); and E's post makes our post ≤ H's post. *)
op P : int → bool.

module type Det = { proc run(x : int) : bool }.

module Prog (D : Det) = {
  proc hit() : bool = { var x, b; x <$ [0..n-1]; b <@ D.run(x); return b; }
}.

module Ref = {
  proc hit() : bool = { var x; x <$ [0..n-1]; return P x; }
}.

ehoare ref_e : Ref.hit : (mu [0..n-1] P)%xr ⟹ res%xr.
proof. by proc; wp; skip ⇒ &hr /=; rewrite Ep_mu. qed.

section.

declare module D <: Det.

declare axiom D_ll : islossless D.run.

declare axiom D_spec (x0 : int) : hoare[D.run : x = x0 ⟹ res = P x0].

equiv prog_ref : Prog(D).hit ~ Ref.hit : true ⟹ ={res}.
proof.
  proc; seq 1 1 : (={x}); first by auto.
  exists* x{1}; elim* ⇒ x0.
  call{1} (: x = x0 ⟹ res = P x0); first by conseq D_ll (D_spec x0).
  by auto.
qed.

ehoare prog_e : Prog(D).hit : (mu [0..n-1] P)%xr ⟹ res%xr.
proof. by conseq prog_ref ref_e ⇒ // &1 &2 →. qed.

end section.
```

16 Pitfalls

- `forall` extends as far to the right as it can. In an invariant write `∧ (forall y, P y) ∧ Q` with the parentheses, or `Q` ends up under the binder.
- `split` and other tactic names cannot be used as lemma or hypothesis names.
- Program expressions cannot mention `glob M`. EasyCrypt reports that expressions cannot contain a glob statement.
- Inside a section, a `declare axiom` cannot use a `local` predicate, and a non-local operator cannot mention the `glob` of a declared module. Spell the predicate out in the axiom.
- `⇒ //` may close more goals than expected, and the next bullet then fails because all goals are closed.

It also removes a `true` precondition: after `bypr (res{1}) (res{2}) ⇒ //` there is no hypothesis left to introduce.

- `ecall{1} (lemma (glob A){1})` crashes with `InvalidGoalShape`. With ordinary program variables `ecall{1}` works, and `ecall{1} (inc_ph P.x{1})` needs no `seq` or `exlim`. Naming the glob first also works: `exlim (glob A){1} ⇒ g; ecall{1} (lemma g)`. Otherwise `seq`, `exlim` and `call{1}` do the same job.
- The order of the side goals from `conseq` is not always the order of its arguments. Look at them before writing bullets.
- `*)` ends a comment wherever it is, so a comment containing `(exists* ... elim*)` closes at `elim*)`.
- `^^` on Booleans needs `require import Bool`, which `AllCore` does not export. `fel` needs `require import FelTactic`.
- A lemma parameter named like a program variable shadows it. In `lemma l (x : int) : hoare[M.f : x = 3 ⟹ ...]` the `x` is the parameter, not the argument of `M.f`. Name parameters `x0` or `xx`.
- Once the section is closed, every lemma of it takes the declared modules and every `declare axiom` declared before it, in order, whether it uses them or not. `print` a lemma to see its statement. Declare an axiom after the lemmas that do not need it, or in a section of its own.
- `sp` runs through an `if` whose branches only assign, and the `if` that follows fails with "invalid first instruction". Give `sp` a count, as in `sp 1; if`. `sp` does not apply to ehoare goals at all; cut with `seq 1 : (inv)`.
- `fel` takes its oracles as the adversary calls them, not the procedures they are aliases of. An entry that names no oracle is silently ignored, and that oracle gets the guard `true`. Its per-oracle goals come in the alphabetical order of the oracles' module names, not in the order of the list.
- In a rewrite chain, `1:/#` is accepted but `1:smt(lemma)` is a parse error. Prove the side fact first with `have`.
- After `clone ... proof *`, a missing `realize` is reported only by the next command, and at the very end of a file not at all: the file compiles. End such a file with a lemma.
- `rename [module] "Atk" as "Coll"` renames every module whose name contains `Atk`, so `AtkJ` becomes `CollJ`.
- EasyCrypt does not check a required file again when it has a fresh `.eco`. Delete the `.eco` files, or check a clean copy, before trusting a build.
- Newer EasyCrypt (after r2026.07) wants the bound of a phoare or an ehoare to be nonnegative, as extra goals of `rnd`, `bypr`, `byehoare` and `exfalso`. Keep `have hb : 0%r ≤ bound` in the context so that `⇒ //` closes them. The files of Sections 10, 11, 14 and 15 work on both versions that way.

17 Seeing the goals from the command line

`easycrypt llm` loads a file up to a line and prints the goals there.

```
easycrypt llm -I <dirs> -eval 'LOAD "file.ec" 42
QUIET ON
proc.
GOALS ALL'
```

- `LOAD` checks everything up to and including the given line, then waits for more input. If that line is `proof. admit. qed.`, split it first, or the load runs past the proof.
- Every further line is one tactic, one sentence per line; two sentences on one line are rejected. A statement that spans several lines cannot be sent: put it in the file and load up to its `proof.` line.
- `GOALS` prints the focused goal, `GOALS ALL` prints all of them, and `TREE ALL` prints the goal tree. An error prints `ERROR` together with the offending `source:` line.